AlphaEval: Evaluating Agents in Production

Pengrui Lu[*1,3,4] Bingyu Xu[*2,6] Wenjun Zhang[*2,5] Shengjia Hua[*2] Xuanjian Gao[2] Ranxiang Ge[2] Lyumanshan Ye[3,4] Linxuan Wu[2] Yiran Li[2] Junfei Fish Yu[9] Yibo Zhang[11] Ruixin Li[11] Manxiang Li[11] Xiao Han[11] Xiaocong Zhou[12] Guangyao Chi[7] Zisheng Chen[8] Kaishen Chen[10] Kun Wang[10] Qihua Xu[10] Fengyue Meng[2] Yuchen Ni[2] Jiajun Li[2] Jinxiu Liu[2] Danfeng Zhang[2] Jingru Zhao[2] Pengfei Liu[†1,3,4]

[1]SII [2]MiraclePlus [3]SJTU [4]GAIR [5]HIT [6]UCAS [7]LangCore [8]Jiqizhixin [9]HunterAI [10]CinoCore [11]KuaFuAI [12]POET

AlphaEval Homepage

Abstract

The rapid deployment of AI agents in commercial settings has outpaced the development of evaluation methodologies that reflect production realities. Existing benchmarks measure agent capabilities through retrospectively curated tasks with well-specified requirements and deterministic metrics—conditions that diverge fundamentally from production environments where requirements contain implicit constraints, inputs are heterogeneous multi-modal documents with information fragmented across sources, tasks demand undeclared domain expertise, outputs are long-horizon professional deliverables, and success is judged by domain experts whose standards evolve over time. We present *AlphaEval*, a production-grounded benchmark of **94** tasks sourced from **seven** companies deploying AI agents in their core business, spanning six O*NET (Occupational Information Network) domains. Unlike model-centric benchmarks, *AlphaEval* evaluates *complete agent products*—Claude Code, Codex, etc.—as commercial systems, capturing performance variations invisible to model-level evaluation. Our evaluation framework covers multiple paradigms (LLM-as-a-Judge, reference-driven metrics, formal verification, rubric-based assessment, automated UI testing, etc.), with individual domains composing multiple paradigms. Beyond the benchmark itself, we contribute a *requirement-to-benchmark construction framework*—a systematic methodology that transforms authentic production requirements into executable evaluation tasks in minimal time. This framework standardizes the entire pipeline from requirement to evaluation, providing a reproducible, modular process that any organization can adopt to construct production-grounded benchmarks for their own domains.

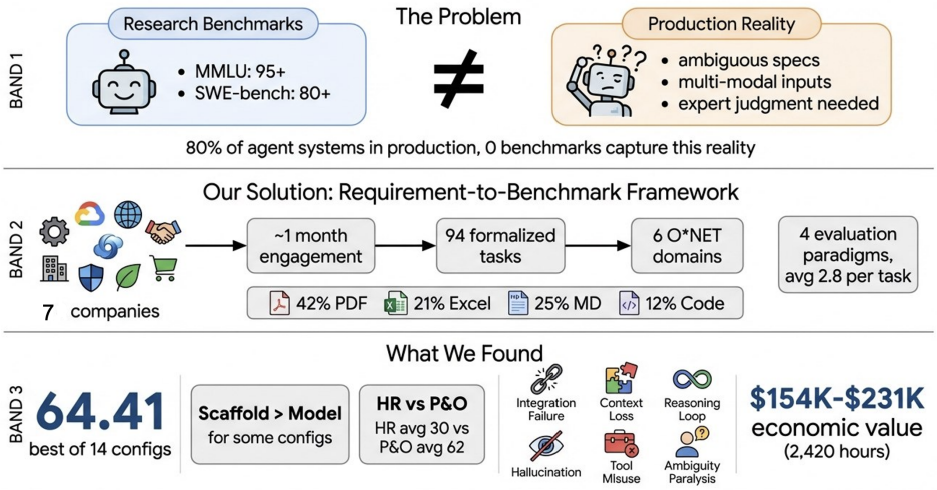

Figure 1: Overview of *AlphaEval*. **Band 1**: The gap between research benchmarks and production reality. **Band 2**: Our requirement-to-benchmark framework transforms production requirements into 94 formalized tasks across 6 O*NET domains. **Band 3**: Key findings—best agent scores 64.41/100, scaffold matters as much as model, and six production-specific failure modes.

* Equal contribution.
† Corresponding author.

1 Introduction

The hardest part of deploying an AI agent is not building the agent—it is knowing whether the agent works. As organizations move from prototypes to production, constructing reliable evaluation infrastructure has emerged as a dominant engineering challenge (Pan et al., 2025). Yet the research community has devoted remarkably little attention to *evaluating agents as they are actually deployed in production*. Existing benchmarks like SWE-bench (Jimenez et al., 2024), WebArena (Zhou et al., 2024), and OSWorld (Xie et al., 2024) have been instrumental in driving agent capabilities forward. However, they share a common design philosophy: tasks are curated retrospectively—drawn from already-completed work artifacts such as resolved GitHub issues or archived web sessions—with well-specified requirements and deterministic metrics, diverging fundamentally from production evaluation. A recent systematic analysis of 43 benchmarks covering 72,342 tasks confirms this mismatch quantitatively, revealing that agent development remains heavily programming-centric and misaligned with the occupational categories where human labor and economic value concentrate (Wang et al., 2026).

Production evaluation operates under different constraints: requirements contain implicit constraints and arrive as loosely worded business descriptions; inputs are heterogeneous multi-modal documents with information fragmented across sources; tasks demand undeclared domain expertise; and success is judged by domain experts' subjective assessments. Among practitioners who reported their deployment stage, over 80% indicated their agent systems are in production or pilot phases (Pan et al., 2025), yet no existing benchmark fully captures this reality. Our survey of 27 AI product companies (Appendix J) quantifies this gap: **63%** of companies report low confidence in whether model updates actually improve their products, 25.9% have *no explicit evaluation criteria*, and 70.4% rely on developers performing testing as a side task. The disconnect is structural along three axes: (1) *Task Under-specification*: benchmarks have explicit goals; production tasks emerge from evolving business needs with implicit constraints invisible to outsiders. (2) *Judgment Subjectivity*: benchmarks evaluate with single-dimension, predefined metrics; production demands multi-faceted quality criteria defined by domain experts. (3) *Continuous Evolution*: benchmarks are built once; production evaluations must continuously evolve.

We address this by partnering with companies deploying AI agents in their core business and AI-focused organizations to capture authentic evaluation scenarios. The result is *AlphaEval*, a benchmark of **94** production-grounded tasks together with a reusable *requirement-to-benchmark construction framework* that standardizes the pipeline from authentic production requirements to fully automated evaluations. We evaluate six frontier models—Claude Opus 4.6, GPT-5.2, Gemini 3 Pro Preview, Kimi K2.5, GLM-5, and MiniMax M2.5—deployed through four commercial agent products: Claude Code, Codex, GitHub Copilot, and Cursor, yielding 14 model–scaffold configurations. Our key findings: (1) the best configuration (Claude Code + Opus 4.6) achieves only 64.41/100, revealing a substantial research-production gap; (2) scaffold choice matters as much as model choice—the same Opus 4.6 model scores 64.41 via Claude Code but only 53.45 via Codex, an 11-point spread; and (3) domain performance varies dramatically, from 62.0 (Technology Research) to 30.0 (Human Resources), indicating that no single aggregate score captures production readiness (Figure 1). Our contributions are fourfold:

- **A requirement-to-benchmark construction framework.** A standardized methodology that transforms authentic production requirements from companies into automated, reproducible evaluation tasks through a modular four-stage pipeline (partner engagement, requirement elicitation, task formalization, iterative validation), enabling systematic and repeatable benchmark construction across diverse professional domains.
- **A production-grounded agent benchmark.** 94 tasks from seven companies classified into six O*NET occupational domains, preserving production complexity—ambiguous specifications, multi-modal inputs, long-horizon deliverables, and stakeholder-aligned evaluation criteria that evolve with business needs.
- **A unified evaluation framework.** Multiple evaluation paradigms through a standardized API with Docker-based sandboxed execution. Individual domains *compose* multiple paradigms, reflecting the multi-dimensional nature of production quality.
- **An empirical analysis of the research-production gap.** Frontier agents exhibit significant degradation on production tasks, with the best configuration achieving only 64.41 on average. We conduct a systematic error analysis of failure cases and provide economic value grounding that translates benchmark scores into dollar-value estimates of professional labor delivered.

2 Preliminaries

Evaluation Methodology Taxonomy. Through systematic analysis of 90+ existing agent-relevant benchmarks, we identify a taxonomy of evaluation methodologies (Figure 2). These span four major paradigms: **Reference**

Evaluation framework available at `https://github.com/GAIR-NLP/AlphaEval`.

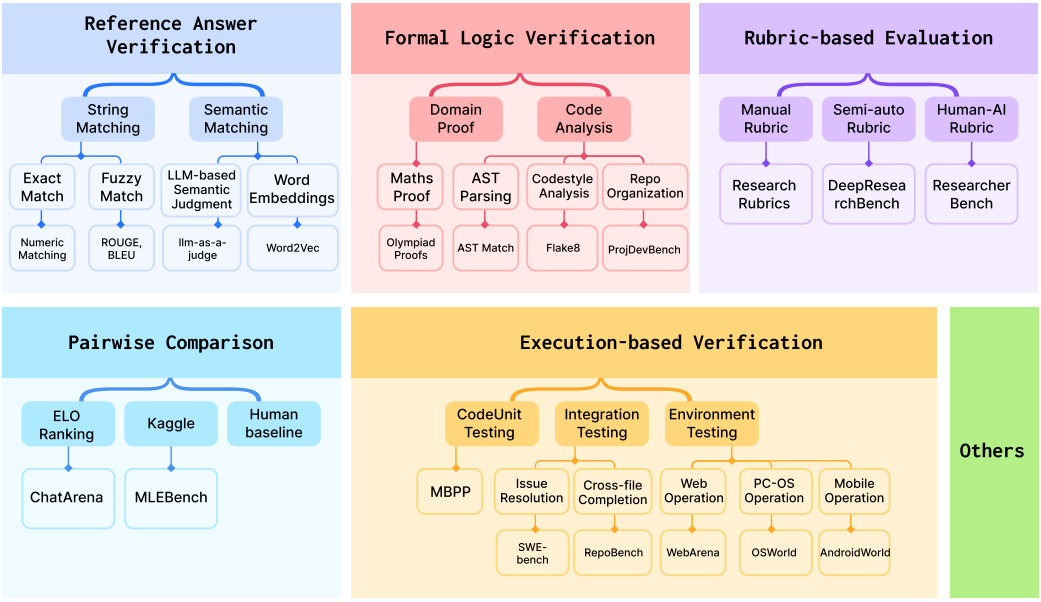

Figure 2: Taxonomy of evaluation methodologies for AI agents. *AlphaEval* covers multiple evaluation paradigms (excluding Pairwise Comparison and Others).

Answer Verification (string and semantic matching against gold answers), **Formal Logic Verification** (mathematical proofs and code analysis), **Rubric-based Evaluation** (manual, semi-automated, or human-AI co-designed scoring rubrics), and **Execution-based Verification** (code unit testing and environment state testing). Beyond these paradigms, LLM-as-a-Judge (Zheng et al., 2023) and Agent-as-a-Judge (Zhuge et al., 2024) serve as cross-cutting evaluation methods that operate *across* paradigms—not merely as semantic matching within Reference Answer Verification, but as flexible judges capable of rubric-based scoring, factual verification, and holistic quality assessment. Notably, most existing benchmarks employ only a single paradigm per task, whereas production evaluation typically requires composing multiple paradigms to capture multi-dimensional quality.

Revisit Existing Agent Benchmarks Task-based agent benchmarks (Jimenez et al., 2024; Zhou et al., 2024; Xie et al., 2024; Li et al., 2026a) enable rigorous capability measurement but curate tasks retrospectively with automatable metrics. Code benchmark surveys (Cao et al., 2025) and protocol-based tool-use benchmarks (Luo et al., 2025) pursue evaluation rigor but remain anchored in research contexts. Domain-specific evaluation for finance (Kuang et al., 2025), e-commerce (Min et al., 2025), and healthcare (Wang et al., 2025a) remain researcher-curated with predetermined scopes. The rise of LLM-as-a-Judge enables scalable evaluation but introduces reliability challenges (Panickssery et al., 2024; Zheng et al., 2023) exacerbated in production settings. Notably, production evaluation spans diverse paradigms—LLM-as-a-Judge, reference-driven metrics, formal verification, rubric-based assessment, and automated UI testing—with most domains combining multiple paradigms, a diversity that existing benchmarks do not capture. Concurrently, Wang et al. (2026) systematically map 43 agent benchmarks to all 1,016 U.S. occupations via O*NET, finding substantial coverage gaps and proposing coverage, realism, and granular evaluation as design principles—criteria that *AlphaEval* operationalizes through production-sourced tasks across six O*NET domains and multi-paradigm evaluation. *AlphaEval* addresses this gap by systematizing production evaluation from companies deploying AI agents, preserving production complexity and contributing both a benchmark and a requirement-to-benchmark construction framework. A comprehensive comparison with 90+ benchmarks across seven dimensions is provided in Table 1.

Table 1: Comprehensive comparison of *AlphaEval* with existing benchmarks across seven dimensions that characterize production-level evaluation: **Production** = tasks sourced from real commercial deployments with paying customers; **Multi-Modal** = requires processing multi-modal heterogeneous inputs (PDFs, spreadsheets, images); **Underspec.** = tasks deliberately preserve production-level requirement ambiguity; **Diverse Eval** = employs three or more distinct evaluation paradigms; **Expert Val.** = evaluation criteria co-developed and validated with domain experts; **Dynamic** = benchmark designed for continuous evolution rather than static one-time release; **Cross-Domain** = spans three or more distinct professional domains.

Benchmark	Domain	Eval Type	Production	Multi-Modal	Underspec.	Diverse Eval	Expert Val.	Dynamic	Cross-Domain
Software Engineering & Coding									
SWE-bench (Jimenez et al., 2024)	Code	Rule	✗	✗	✗	✗	✗	✗	✗
SWE-bench Multimodal (Yang et al., 2024)	Code	Rule	✗	✓	✗	✗	✗	✗	✗
Multi-SWE-bench (Zan et al., 2025)	Code	Rule	✗	✗	✗	✗	✗	✓	✗
SWE-Lancer (Miserendino et al., 2025)	Code	Rule	✓	✗	✗	✗	✗	✗	✗
SWT-Bench (Mündler et al., 2024)	Code	Rule	✗	✗	✗	✗	✗	✗	✗
Terminal-bench (Merrill et al., 2026)	Code	Rule	✗	✗	✗	✗	✗	✓	✗
FeatBench (Chen et al., 2025b)	Code	Rule	✗	✗	✗	✗	✗	✓	✗
DevBench (Li et al., 2024)	Code	R+M+H	✗	✗	✗	✓	✗	✗	✗
LongCLI-Bench (Feng et al., 2026)	Code	Rule	✗	✗	✗	✗	✗	✗	✗
ProjDevBench (Lu et al., 2026)	Code	Rule+Model	✗	✗	✗	✗	✗	✗	✗
Data Science & ML Engineering									
DSBench (Jing et al., 2024)	Code	Model	✗	✓	✗	✗	✗	✗	✗
MLE-bench (Chan et al., 2024)	Code	Rule	✗	✗	✗	✗	✗	✗	✗
KernelBench (Ouyang et al., 2025)	Code	Rule	✗	✗	✗	✗	✗	✗	✗
DAComp (Lei et al., 2025)	Code+Research	Rule+Model	✗	✗	✗	✗	✗	✗	✗
Code Competition & Security									
LiveCodeBench (Jain et al., 2024)	Code	Rule	✗	✗	✗	✓	✗	✓	✗
CodeElo (Quan et al., 2025)	Code	Rule	✗	✗	✗	✗	✗	✗	✗
Aider Polyglot (Gauthier, 2024)	Code	Rule	✗	✗	✗	✗	✗	✗	✗
CyBench (Zhang et al., 2024)	Code	Rule	✗	✗	✗	✗	✗	✗	✗
BountyBench (Zhang et al., 2025)	Code	Rule	✗	✗	✗	✗	✗	✗	✗
VimGolf-Gym (Cybergod AGI Research, 2025)	Code	Rule	✗	✗	✗	✗	✗	✗	✗
DPAI Arena (JetBrains, 2025)	Code	Rule	✗	✗	✗	✗	✗	✓	✗
Spring AI Bench (Spring Community, 2025)	Code	Rule	✗	✗	✗	✗	✗	✗	✗
AGENTS.md Eval (Gloaguen et al., 2026)	Code	Rule	✗	✗	✗	✗	✗	✗	✗
Tool Use & Web Interaction									
WebArena (Zhou et al., 2024)	Web	Rule+Model	✗	✗	✗	✗	✗	✗	✓
AgentBench (Liu et al., 2023)	Tool	Rule	✗	✗	✗	✗	✗	✗	✓
AgentBoard (Ma et al., 2024)	Tool	Rule	✗	✗	✗	✗	✗	✗	✓
τ-bench (Yao et al., 2024)	Tool	Rule	✗	✗	✗	✗	✗	✗	✗
τ^2-Bench (Barres et al., 2025)	Tool	Rule	✗	✗	✗	✗	✗	✗	✗
TheAgentCompany (Xu et al., 2024)	Tool	Rule+Model	✗	✓	✗	✗	✗	✗	✗
Tool Decathlon (Li et al., 2025)	Tool	Rule	✗	✗	✗	✗	✗	✗	✓
ACEBench (Chen et al., 2025a)	Tool	Rule+Model	✗	✗	✗	✓	✗	✗	✓
MCP-Universe (Luo et al., 2025)	Tool	Rule	✗	✗	✗	✗	✗	✗	✓
BFCL (Patil et al., 2025)	Tool	Rule	✗	✗	✗	✗	✗	✓	✓
Context-Bench (Letta, 2025a)	Tool	Rule	✗	✗	✗	✗	✗	✓	✗
Letta Evals (Letta, 2025b)	Tool	Rule+Model	✗	✗	✗	✗	✗	✓	✗
EcomBench (Min et al., 2025)	Tool	Model	✗	✗	✗	✗	✓	✓	✗
DeliveryBench (Mao et al., 2025)	Tool	Rule	✗	✓	✗	✗	✗	✗	✗
WorFBench (Qiao et al., 2024)	Tool	Rule	✗	✗	✗	✗	✗	✗	✓
BrowseComp (Wei et al., 2025)	Search	Model	✗	✗	✗	✗	✗	✗	✓
AgencyBench (Li et al., 2026a)	Code+Tool	Rule+Model	✗	✗	✗	✗	✓	✗	✓
HammerBench (Wang et al., 2024)	Tool	Rule	✗	✗	✗	✗	✗	✗	✗
Operating System & GUI									
GAIA (Mialon et al., 2023)	OS	Rule	✗	✓	✗	✗	✗	✗	✓
OSWorld (Xie et al., 2024)	OS	Rule	✗	✓	✗	✗	✗	✗	✗
AppWorld (Trivedi et al., 2024)	OS	Rule	✗	✗	✗	✗	✗	✗	✗
WebSuite (Li and Waldo, 2024)	OS	Rule	✗	✗	✗	✗	✗	✗	✗
OSUniverse (Davydova et al., 2025)	OS	R+M+H	✗	✓	✗	✓	✗	✗	✗
OdysseyBench (Wang et al., 2025b)	OS	Rule	✗	✗	✗	✗	✗	✗	✗
OfficeQA (Opsahl-Ong et al., 2026)	OS	Rule	✗	✓	✗	✗	✗	✗	✗
Scientific Research									
EXP-Bench (Kon et al., 2025)	Research	Model	✗	✗	✗	✗	✗	✗	✗
PaperBench (Starace et al., 2025)	Research	Model	✗	✗	✗	✗	✓	✗	✗
CORE-Bench (Siegel et al., 2024)	Research	Rule	✗	✓	✗	✗	✗	✗	✓
Auto-Bench (Chen et al., 2025c)	Research	Rule	✗	✗	✗	✗	✗	✗	✗

Continued on next page

Table 1 continued from previous page

Benchmark	Domain	Eval Type	Production	Multi-Modal	Underspec.	Diverse Eval	Expert Val.	Dynamic	Cross-Domain
ResearchCodeBench (Hua et al., 2025)	Research	Rule	✗	✗	✗	✗	✗	✓	✗
AstaBench (Bragg et al., 2025)	Research	Rule+Model	✗	✗	✗	✗	✗	✗	✓
AInsteinBench (Duston et al., 2025)	Code+Research	Rule	✗	✗	✗	✗	✗	✗	✓
ResearchGym (Garikaparthi et al., 2026)	Research+Code	Rule	✗	✗	✗	✗	✗	✗	✗
Mathematics & Knowledge									
MMLU (Hendrycks et al., 2020)	Knowledge	Rule	✗	✗	✗	✗	✗	✗	✓
GPQA Diamond (Rein et al., 2023)	Knowledge	Rule	✗	✗	✗	✗	✓	✗	✓
MMMU (Yue et al., 2023)	Knowledge	Rule	✗	✓	✗	✗	✗	✗	✓
MathVista (Lu et al., 2024)	Math	Rule	✗	✓	✗	✗	✗	✗	✗
FrontierMath (Glazer et al., 2024)	Math	Rule	✗	✗	✗	✗	✓	✗	✗
AIME (MAA, 2025)	Math	Rule	✗	✗	✗	✗	✗	✓	✗
HMMT (Harvard-MIT, 2025)	Math	Rule	✗	✗	✗	✗	✗	✓	✗
USAMO (Petrov et al., 2025)	Math	Human	✗	✗	✗	✗	✓	✓	✗
MMMLU (OpenAI, 2024)	Knowledge	Rule	✗	✗	✗	✗	✗	✗	✓
Video-MME (Fu et al., 2024)	Knowledge	Rule	✗	✓	✗	✗	✗	✗	✓
OpenAI-MRCR (OpenAI, 2025)	Knowledge	Rule	✗	✗	✗	✗	✗	✗	✗
HLE (Phan et al., 2025)	Knowledge	Rule	✗	✓	✗	✗	✓	✗	✓
ARC-AGI-2 (Chollet and ARC Prize Team, 2025)	Reasoning	Rule	✗	✗	✗	✗	✗	✗	✗
OODBench (Lin et al., 2026)	Knowledge	Rule	✗	✓	✗	✗	✗	✗	✗
Agent Product Evaluation									
xbench (xbench Team, 2025)	Recruit+Mkt	Model	✓	✗	✗	✗	✓	✓	✗
AgentIF-OneDay (AgentIF Team, 2026)	Daily Tasks	Model	✗	✓	✗	✗	✓	✗	✓
Emerging Benchmarks (2026)									
Persona2Web (Kim et al., 2026a)	Search	Model	✗	✗	✓	✗	✗	✗	✗
AmbiBench (Sun et al., 2026)	OS	Model	✗	✓	✓	✗	✗	✗	✗
PAHF (Liang et al., 2026)	Tool	Rule+Model	✗	✗	✗	✗	✗	✗	✗
AgenticShop (Kim et al., 2026b)	Search	Model	✗	✗	✗	✗	✗	✗	✗
GAP Benchmark (Cartagena and Teixeira, 2026)	Tool	Rule	✗	✗	✗	✗	✗	✗	✓
AgentLAB (Jiang et al., 2026)	Safety	Model	✗	✗	✗	✗	✗	✗	✓
STING (Talokar et al., 2026)	Safety	Model	✗	✗	✗	✗	✗	✗	✗
GT-HarmBench (Cobben et al., 2026)	Safety	Rule	✗	✗	✗	✗	✗	✗	✗
ForesightSafety (Tong et al., 2026)	Safety	Model	✗	✗	✗	✗	✗	✗	✓
APST (Broadwater, 2026)	Safety	Model	✗	✗	✗	✗	✗	✗	✗
MemoryArena (He et al., 2026)	Search+Research	Rule	✗	✗	✗	✗	✗	✗	✗
WebWorld-Bench (Xiao et al., 2026)	Search+Code	Rule	✗	✗	✗	✗	✗	✗	✓
Gaia2 (Froger et al., 2026)	OS+Search	Rule	✗	✗	✗	✗	✗	✗	✗
SkillsBench (Li et al., 2026b)	Code+Tool	Rule	✗	✗	✗	✗	✗	✗	✓
MATEO (Roccabruna et al., 2026)	Reasoning	Rule	✗	✓	✗	✗	✗	✗	✗
SciAgentGym (Shen et al., 2026)	Research+Tool	Rule+Model	✗	✗	✗	✗	✗	✗	✓
Drug Scouting (Vinogradova et al., 2026)	Search+Research	Model	✗	✗	✗	✗	✗	✗	✗
AD-Bench (Hu et al., 2026)	Tool	Rule	✓	✗	✗	✗	✓	✗	✗
GUI-GENESIS (Cao et al., 2026)	OS	Rule	✗	✓	✗	✗	✗	✗	✗
BookingArena (Logeswaran et al., 2026)	Search	Rule+Model	✗	✗	✗	✗	✗	✗	✗
BrowseComp-V^3 (Zhang et al., 2026)	Search	Rule	✗	✓	✗	✗	✓	✗	✗
Collective Behavior (Willis et al., 2026)	Research	Rule	✗	✗	✗	✗	✗	✗	✗
Unsafer (Li et al., 2026c)	Tool	Model	✗	✗	✗	✗	✗	✗	✗
Proxy State Eval (Chuang et al., 2026)	Tool	Model	✗	✗	✗	✗	✗	✗	✗
***AlphaEval* (Ours)**	**6 Domains**	**Rule+Model**	✓	✓	✓	✓	✓	✓	✓

3 From Production Requirements to Executable Benchmarks

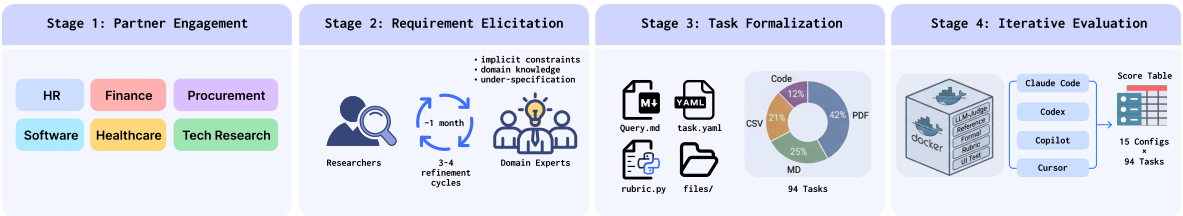

Figure 3: The requirement-to-benchmark construction framework: four stages from production requirements to automated evaluation.

Existing benchmarks are constructed retrospectively: researchers select artifacts (e.g., resolved GitHub issues), then design evaluation criteria around them. This approach yields clean, reproducible tasks but fundamentally

Domain (O*NET)	Representative Task	Tasks	Input	Eval Paradigm
Human Resources (13-1071)	Resume screening vs. JD	11	PDF, JPEG	F1 score
Finance & Investment (13-2051)	Segment research & pitch critique	22	PDF, MD, TXT	LLM-as-a-Judge
Procurement & Operations (13-1020)	BOM cost optimization	23	Excel, CSV, MD	Constraint verif.
Software Engineering (15-1252)	Full-stack app generation	11	YAML, Code	UI testing
Healthcare & Life Sci. (29-9099)	eCRF & insurance policy analysis	16	PDF, MD	LLM + Numerical
Technology Research (15-1221)	AI industry deep analysis	11	MD	LLM-as-a-Judge
Total		**94**		

Table 2: Overview of *AlphaEval* task domains classified by O*NET occupational taxonomy (U.S. Department of Labor, 2025), where Eval Paradigm indicates the primary evaluation method.

cannot capture the under-specification, implicit constraints, and domain expertise that characterize production work. *AlphaEval* inverts this direction: we start from *authentic production requirements*—the actual tasks companies need AI agents to perform for paying customers—and systematically transform them into executable, automated evaluations.

This requirement-to-benchmark construction framework is itself a central contribution. The key challenge is not building a benchmark *perse*, but providing a standardized, repeatable process from a real-world production need to a rigorous, reproducible evaluation. This enables evaluation of agents at a fundamentally deeper level—not just *can the agent code?* but *can the agent deliver value on the actual tasks businesses pay for?* Below we describe each stage of this framework (Figure 3).

Partner Engagement. We partner with companies whose professional workflows intersect with AI agent capabilities, targeting diversity in O*NET occupational domains (Table 2). Partners include both companies deploying AI agents as core products for paying customers and domain-expert organizations whose daily workflows generate authentic tasks suitable for agent evaluation (e.g., technology media producing in-depth industry analysis). We select partners that satisfy as many of the following criteria as possible: (1) access to authentic, professionally validated task requirements that yield long-horizon deliverables (complete reports, codebases, or analytical artifacts rather than short answers); (2) AI agents integral to revenue-generating workflows; (3) diverse input modalities; (4) domain expertise to co-design stakeholder-aligned evaluation criteria and iteratively refine them as business needs evolve; (5) willingness to share anonymized data.

Requirement Elicitation. For each company, we conduct structured engagement spanning approximately one month, combining both online meetings and on-site visits. The critical insight is that production requirements rarely arrive as well-specified task descriptions—they emerge through iterative dialogue. A typical engagement proceeds through three phases: (1) *Workflow discovery*—companies demonstrate their end-to-end workflows, often revealing that the actual task complexity far exceeds initial descriptions (e.g., a clinical data partner initially described their need as "converting Word documents to JSON," but meetings revealed a four-layer reasoning chain: temporal phase identification, trigger rule extraction, form field mapping, and constraint validation); (2) *Scope negotiation*—we jointly determine which segments of long production pipelines can be isolated into self-contained evaluation tasks while remaining professionally meaningful (e.g., negotiating whether to evaluate the full end-to-end CRF construction or focus on visit window computation, which captures the core reasoning challenge); and (3) *Ground truth co-construction*—domain experts provide or validate reference outputs, as standard answers often do not exist in pre-defined form and must be extracted from actual business decisions (e.g., using real interview shortlists rather than AI-generated candidate rankings). Through at least weekly meetings, we jointly develop task specifications that preserve the original level of under-specification, implicit constraints, and domain knowledge dependency that make production tasks fundamentally harder than research tasks.

Task Formalization. We provide partner companies with our standardized task package format upfront, and they deliver their tasks according to this structure—ensuring consistency across domains while allowing domain-specific flexibility in evaluation design. Each task is formalized into a self-contained package: (1) **Task Specification** (`query.md`)—natural language description preserving the original level of specification; (2) **Task Configuration** (`task.yaml`)—structured metadata including task name, domain category, difficulty level, evaluation type, and agent timeout; (3) **Input Files** (`files/`)—raw documents required by the task (PDFs, Excel, images, etc.); (4) **Evaluation Specification** (`.eval/rubric.py`)—composing one or more evaluation paradigms, with optional `ground_truth.json` as reference answers. This format serves as a shared contract between our team and partners: we define the structure, partners populate it with authentic production content. Approximately 42% of tasks involve PDFs, 21% structured data files, 25% markdown/text, and 12% code/YAML.

Iterative Validation. We validate evaluation tasks using frontier agents internally and through collaborative verification with partner companies, averaging three to four refinement cycles per company. This iterative process ensures stakeholder-aligned evaluation: rubrics faithfully capture the quality dimensions that matter to paying customers—not just technical correctness, but the holistic deliverable quality that determines whether a business would accept the agent's output. Crucially, these criteria are not static: as agent capabilities improved during our evaluation period, several partners raised their quality bars, reflecting the dynamic nature of production evaluation standards.

4 The *AlphaEval* Benchmark

AlphaEval comprises **94** tasks sourced from companies with paying customers, classified into six O*NET occupational domains (summary statistics in Table 8). Several characteristics distinguish it from existing benchmarks: *production provenance* (every task from an active commercial deployment), *implicit constraints* (requirements contain hidden rules and undeclared priorities invisible to outsiders), *information fragmentation* (key information is scattered across different locations in multiple documents, requiring cross-document reasoning), *domain knowledge dependency* (tasks demand expertise beyond the task specification, such as medical insurance policies, investment analysis frameworks, and GCP standards), *multi-modal heterogeneity* (agents process PDFs, spreadsheets, scanned images, and code within single tasks), *long-horizon deliverables* (outputs are complete professional artifacts—10-page investment reports, full-stack codebases, multi-visit clinical calculations—rather than short answers or code snippets), *stakeholder-aligned and evolving evaluation* (evaluation criteria are co-designed with domain practitioners who represent actual customers, and may shift as business requirements and agent capabilities evolve), and *evaluation pluralism* (covers multiple evaluation paradigms).

Task Categories. We classify tasks following the O*NET occupational taxonomy (U.S. Department of Labor, 2025), ensuring each domain maps to a standardized work activity category. This classification enables systematic coverage analysis and makes the benchmark extensible—any new production task can be mapped to an existing O*NET domain.

Human Resources (11 tasks, O*NET 13-1071): Agents screen candidate resumes against job descriptions. A representative task provides 24 resumes (PDF and JPEG) for an AI Scientist internship at a startup accelerator, mixing objective criteria ("at least one internship longer than 3 months") with subjective ones ("passion for cutting-edge technology"). The agent must select exactly 6 candidates; evaluation computes F1 against actual interview decisions.

Finance & Investment (22 tasks, O*NET 13-2051): Agents perform investment research and financial data extraction. Tasks include: (a) generating professional segment research reports from startup business plans, following prescribed templates covering market sizing (TAM-SOM-SAM), competitive landscape, and technology deep-dive (14 tasks); (b) synthesizing unstructured meeting transcripts into actionable investor critiques—e.g., designing a 2-minute pitch storyline and assessing whether a healthcare platform's 20,000-user reach translates to genuine product-market fit given only 70 core users (5 tasks); and (c) extracting structured financial data from multi-year corporate annual reports (3 tasks). Evaluation combines LLM-as-a-Judge with structural validation.

Procurement & Operations (23 tasks, O*NET 13-1020): Agents solve constrained optimization and data processing problems grounded in real procurement workflows. A representative task presents 2,000 board cards with specifications and pricing in Excel, plus a natural language requirements document with implicit constraints (20 tasks). Additional tasks involve procurement bidding data analysis and spreadsheet operations (3 tasks). Evaluation programmatically verifies constraint satisfaction and cost optimality.

Software Engineering (11 tasks, O*NET 15-1252): Agents build or modify full-stack mobile applications from detailed product requirements. A representative task provides database schemas, seed data, and a 200-line requirements document for a poetry appreciation app (built with the UniApp framework) featuring AI-powered playback, community posting, and collection management across four navigation pages. Evaluation employs automated end-to-end UI testing: a headless browser executes user flows and scores 10+ functional requirements.

Healthcare & Life Sciences (16 tasks, O*NET 29-9099): Agents handle clinical trial management and healthcare policy analysis. Tasks include: (a) computing visit windows for electronic Case Report Form (eCRF) systems, where shifting one visit propagates through subsequent windows via cascade dependencies (10 tasks); and (b) analyzing pharmaceutical reimbursement policies, medical insurance calculations, and drug coverage regulations (6 tasks). Evaluation combines LLM-as-a-Judge with numerical verification.

Technology Research (11 tasks, O*NET 15-1221): Agents conduct comprehensive investigations on current technology topics, requiring web search, multi-source synthesis, and structured report generation. A representative task asks the agent to research the survival status of AI agent startups that raised over $100M, including failures, acquisitions, and emerging trends. Evaluation uses LLM-as-a-Judge with weighted rubric points requiring source-backed evidence.

Evaluation Methodologies. *AlphaEval* covers four of the five major evaluation paradigms identified in our taxonomy (Section 2): **Reference Answer Verification**, **Formal Logic Verification**, **Rubric-based Evaluation**, and **Execution-based Verification** (detailed per-paradigm coverage in Table 7). We additionally employ LLM-as-a-Judge (Claude Opus 4.6) as a cross-cutting semantic evaluation method within multiple paradigms. Critically, individual domains *compose* multiple paradigms—e.g., clinical research combines LLM-as-a-Judge with numerical verification. Every task employs $\geq$2 leaf-node evaluation types (avg. 2.8; Table 9), reflecting the multi-dimensional nature of production quality. All rubric scripts output standardized scores (0.0–1.0) enabling cross-domain comparison.

Economic Value Annotation. To ground task difficulty in economic terms, we annotate each task with its *human replacement cost* through a two-stage pipeline: automated AI estimation followed by domain expert calibration (full methodology in Appendix G). After calibration, the 94 tasks represent **2,420 professional hours** ($\sim$60 person-weeks) of labor, valued at **$154K–$231K** (USD) or **¥391K–¥570K** (CNY)—validating that *AlphaEval* captures economically meaningful work.

Evaluation Infrastructure. The *AlphaEval* framework is organized around three abstractions: a **Task Runner** managing evaluation lifecycle, an **Evaluator Registry** routing tasks to paradigm-specific pipelines, and an **Execution Sandbox** using Docker containers for isolation. Each rubric script outputs a standardized score $s_{\text{task}} = \sum_k w_k \cdot e_k \in [0, 1]$, where e_k is the evaluation result from paradigm k with expert-assigned weight w_k. Domain scores are unweighted task means; the overall score is the unweighted mean across six domains, giving each domain equal influence regardless of task count (see Appendix B for details).

Challenges in Production Evaluation. Constructing *AlphaEval* surfaced challenges largely absent from research benchmark development: (1) *Preserving productive ambiguity*—fully disambiguating tasks destroys what makes production tasks difficult; (2) *Implicit constraints*—practitioners consider requirements "obvious" that are absent from written specifications; (3) *Quantifying subjective judgment*—experts assess quality holistically, and decomposing into verifiable dimensions inevitably loses information; (4) *Evaluation criteria drift*—because evaluation is stakeholder-aligned, partner companies revised quality standards as agent capabilities improved and business priorities shifted, requiring ongoing rubric maintenance; (5) *Environment fidelity*—production agents operate within rich software environments difficult to reproduce in sandboxed evaluation; and (6) *Balancing openness and confidentiality*—partner companies are cautious about sharing evaluation criteria encoding competitive knowledge.

5 Experiments

5.1 Setup

Models. We evaluate six frontier models spanning both closed-source and open-source ecosystems. *Closed-source models*: Claude Opus 4.6 (Anthropic), GPT-5.2 (OpenAI), and Gemini 3 Pro Preview (Google)—representing the strongest proprietary offerings from three major AI labs. *Open-source models*: Kimi K2.5 (Moonshot), GLM-5 (Zhipu AI), and MiniMax M2.5 (MiniMax)—representing competitive open-weight alternatives. This selection enables direct comparison of the open-source vs. closed-source performance gap in production settings.

Agent Systems. Models are deployed through four commercial agent products: **Claude Code** (Anthropic), **Codex** (OpenAI), **GitHub Copilot** (GitHub), and **Cursor** (Cursor). All agents are invoked via their respective CLI interfaces within Docker-sandboxed environments with pinned versions for reproducibility (Table 6 in Appendix). We record full output trajectories (tool calls, intermediate reasoning, and final artifacts) for each run, enabling post-hoc failure analysis.

Configurations. While $6 \times 4 = 24$ combinations are theoretically possible, we evaluate 14 configurations (Table 3), selecting combinations based on two criteria: (1) *real-world adoption*—we prioritize model–scaffold pairings that are widely used in production settings by our partner companies and research teams (e.g., Claude Code + Opus, Codex + GPT-5.2, Cursor + Opus); and (2) *evaluation cost*—each full benchmark run involves 94 tasks with substantial agent execution (e.g., Claude Code + Opus averages 46 turns and 14 minutes per task), making exhaustive enumeration prohibitively expensive. The selected 14 configurations cover all four scaffolds and all six models, ensuring comprehensive coverage while focusing on practically relevant pairings.

5.2 Results and Domain-Level Analysis

Several key findings emerge from Table 3:

- **Low absolute scores.** The best configuration (Claude Code + Opus 4.6) achieves only 64.41 on average, revealing a substantial research-production gap.

Agent Product	Model	HR (11)	F&I (22)	P&O (23)	SE (11)	H&LS (16)	TR (11)	Avg.	Value (USD K)
Claude Code	Claude Opus 4.6	35.91	**70.35**	83.35	**70.95**	**50.06**	75.82	**64.41**	**110–165**
	Gemini 3 Pro	26.73	54.68	65.39	59.47	39.88	58.55	50.78	89–133
	GPT-5.2	26.55	38.64	40.22	55.08	27.94	48.36	39.47	70–106
	Kimi K2.5	20.18	47.80	35.04	56.77	44.44	59.18	43.90	78–116
	GLM-5	31.91	51.18	66.43	52.46	38.88	51.36	48.70	82–122
	MiniMax M2.5	26.00	52.55	39.39	42.03	34.62	50.73	40.89	70–105
Codex	GPT-5.2	27.36	62.80	52.35	50.56	32.19	60.27	47.59	84–126
	GLM-5	24.36	57.85	56.74	53.58	46.69	59.91	49.85	85–128
	Claude Opus 4.6	32.18	45.99	79.00	54.74	36.81	72.00	53.45	86–129
	Kimi K2.5	35.36	47.27	30.91	52.52	38.75	53.73	43.09	73–109
GitHub Copilot	GPT-5.2	32.00	63.39	74.04	61.07	30.69	68.27	54.91	97–145
	Gemini 3 Pro	28.27	59.31	65.35	50.46	30.69	65.45	49.92	86–128
	Claude Opus 4.6	34.18	62.12	**88.09**	63.07	44.06	**76.36**	61.31	102–153
Cursor	Claude Opus 4.6	**38.91**	67.10	**88.09**	64.77	44.06	68.18	61.85	104–156

Table 3: Average scores (0–100) on *AlphaEval* across six O*NET domains. Value (USD K) estimates the human replacement cost each configuration delivers, computed as $\sum_d \text{score}_d \times \text{domain_value}_d$ using per-domain expert-calibrated costs (Table 10). **Bold**: best in column. Underline: second best.

- **Scaffold matters as much as model.** The same Claude Opus 4.6 scores 64.41 via Claude Code, 61.85 via Cursor, 61.31 via GitHub Copilot, but only 53.45 via Codex. GPT-5.2 scores 39.47 via Claude Code but 54.91 via GitHub Copilot—a 15-point spread. This confirms that evaluating *agent systems*, not just models, is essential.
- **Extreme domain variance.** Procurement & Operations scores range from 30.91 to 88.09, while Human Resources scores never exceed 38.91—no single aggregate score captures production readiness. Model rankings are also domain-dependent: GLM-5 scores 66.43 on Procurement & Operations but only 52.46 on Software Engineering, meaning aggregate rankings would be misleading.
- **Score ranking $\neq$ value ranking: implications for agent selection.** Domain-weighted economic value reveals a different picture from average scores. Codex + Opus 4.6 (avg. 53.45, \$86K–\$129K) outscores Claude Code + Gemini 3 Pro (avg. 50.78) but delivers *less* economic value (\$89K–\$133K), because Gemini 3 Pro performs better on high-value domains (Software Engineering, Finance & Investment). This has direct implications for agent selection in practice: (1) organizations should select configurations based on their *domain portfolio*—a company primarily doing financial analysis should weight F&I performance heavily, not rely on aggregate scores; (2) a *multi-agent strategy* may be optimal, routing different task types to different configurations (e.g., Claude Code + Opus for finance and research, Copilot + Opus for procurement); and (3) the \$40K–\$60K value gap between configurations provides organizations with a quantitative basis for agent selection decisions, grounding what is typically an intuition-driven choice in concrete economic terms.

We order domains from easiest to hardest (highest to lowest average score across configurations): **Technology Research** (avg. 62.0): scores range from 48.36 to 76.36, with strong performance from configurations using Opus 4.6. Tasks requiring up-to-date information retrieval, multi-source synthesis, and technical depth remain challenging, but agents with persistent search strategies achieve meaningful scores. **Procurement & Operations** (avg. 61.7): Cursor + Opus 4.6 and Copilot + Opus 4.6 reach 88.09; binary pass/fail scoring on the core optimization tasks reflects the zero-tolerance nature of procurement decisions. **Software Engineering** (avg. 56.3): even weaker models exceed 42 points, but the gap between top (70.95) and bottom (42.03) remains substantial. **Finance & Investment** (avg. 55.8): Claude Code + Opus 4.6 leads at 70.35, with strong performance from Codex configurations that achieve near-perfect scores on financial data extraction subtasks. **Healthcare & Life Sciences** (avg. 38.6): Claude Code + Opus 4.6 leads at 50.06; clinical trial eCRF tasks exhibit zero tolerance for numerical errors, while healthcare policy tasks require domain-specific regulatory knowledge. **Human Resources** (avg. 30.0): best score only 38.91—aligning agent judgments with human hiring decisions remains difficult.

Economic Value Delivered. Translating scores into economic terms using per-domain expert-calibrated labor costs (Appendix G, Table 10), the best configuration delivers an estimated **\$110K–\$165K** in professional labor value while the worst delivers \$70K–\$105K—a **\$40K–\$60K gap** from the same 94 tasks. The Value column in Table 3 reports these domain-weighted values for all 14 configurations, providing organizations with a quantitative basis for agent selection beyond aggregate scores alone.

5.3 Evaluation Reliability

Statistical Reliability. To address single-run stochasticity, we conduct repeated evaluations for our best-performing configuration (Claude Code + Opus 4.6) across three independent runs (Table 4). The narrow confidence intervals (overall ± 1.83) confirm that reported scores are stable and that configuration rankings are reproducible across runs. Variance naturally differs across evaluation paradigms—constraint-verification domains show higher variance (P&O std=4.72) than LLM-as-a-Judge domains (F&I std=1.87, TR std=3.56)—reflecting the inherent characteristics of each paradigm rather than evaluation instability.

Domain	Mean	Std	95% CI
Human Resources	35.91	2.14	[33.77, 38.05]
Finance & Investment	70.35	1.87	[68.48, 72.22]
Procurement & Ops.	83.35	4.72	[78.63, 88.07]
Software Engineering	70.95	3.21	[67.74, 74.16]
Healthcare & Life Sci.	50.06	2.93	[47.13, 52.99]
Technology Research	75.82	3.56	[72.26, 79.38]
Overall	**64.41**	**1.83**	**[62.58, 66.24]**

Table 4: Repeated evaluation results for Claude Code + Opus 4.6 (3 runs). CI = 95% confidence interval.

Meta-Evaluation. We validate evaluation reliability on 20 randomly sampled LLM-as-a-Judge tasks across all six domains (5 configurations, 1,000 rubric point judgments). Two independent expert annotators (strict A, lenient B) assess each rubric point alongside the automated judge (Table 5).

Pair	Agr.	κ	ρ	r
A vs. B	84.7%	0.691	0.818	0.870
A vs. LLM-as-a-Judge	85.0%	0.697	0.820	0.861
B vs. LLM-as-a-Judge	89.7%	0.780	0.845	0.885
Three-way (Fleiss)	79.7%	0.720	—	—

Table 5: Meta-evaluation results: pairwise agreement between two expert annotators (A, B) and the automated LLM-as-a-Judge evaluation across 1,000 rubric point judgments.

All pairwise Cohen's κ values fall within the *substantial agreement* range (0.69–0.78), with Fleiss' $\kappa = 0.720$ confirming three-way reliability. The automated judge shows higher agreement with the lenient annotator ($\kappa = 0.780$) than the strict one ($\kappa = 0.697$), consistent with known LLM-as-a-Judge evaluation biases such as self-preference (Panickssery et al., 2024) and self-enhancement (Zheng et al., 2023).

5.4 Failure Mode Analysis

Beyond aggregate performance gaps, we conduct qualitative analysis to understand *why* agents fail on production tasks. We identify six production-specific failure modes invisible to coding benchmarks (detailed case studies in Appendix H and H.2):

(1) Cascade dependency failure. In Healthcare & Life Sciences, misidentifying a Day 1 anchor produces systematically incorrect window calculations for all subsequent visits. In Finance & Investment, misidentifying a company's industry cascades through market sizing, comparable company selection, and competitive analysis.

(2) Subjective judgment collapse. In Human Resources, agents extract factual qualifications but fail on soft-skill inference. Tasks with quantifiable criteria score 2–3$\times$ higher than those requiring holistic judgment.

(3) Information retrieval failures. Technology Research tasks expose five cognitive failure modes: factual hallucination ($\sim$30%), imprecise retrieval ($\sim$35%), rigid search strategies ($\sim$15%), attribution confusion ($\sim$10%), and positive-information bias ($\sim$10%). For example, Claude Opus 4.6 substitutes outdated Series D data for Harvey AI's F-round with every data point incorrect by a large margin and no hedging language (Table 11 in Appendix). Models also systematically miss negative events (startup failures, funding collapses) because default searches surface success stories—*no model* reported Robin AI's distressed sale despite it being a prominent Series C failure.

(4) Cross-section logical inconsistency. In Finance & Investment, agents produce individually plausible paragraphs that contradict each other—a TAM of \$50B in one section but \$80B two pages later. Models lack a global coherence mechanism across long-form outputs.

(5) Constraint misinterpretation. In Procurement & Operations, agents optimize explicitly stated objectives while violating implicit constraints, and exhibit "synergy blindness"—optimizing components independently rather

than jointly. When a procurement problem has no feasible solution (conflicting constraints), the majority of agent responses fabricate a "best effort" solution rather than declaring infeasibility—a particularly dangerous behavior in production.

(6) Format compliance failures. The most production-specific failure: agents produce substantively reasonable analyses that score poorly because the output format is incompatible with downstream consumption.

6 Discussion

AlphaEval demonstrates that production-grounded evaluation reveals capability gaps invisible to research benchmarks—not merely harder tasks, but a qualitative mismatch between the skills research benchmarks select for (precise instruction following, deterministic reasoning, short-horizon outputs) and what production demands (tolerance for ambiguity, domain-appropriate judgment, long-horizon deliverables, and format compliance under stakeholder-defined quality standards). For model developers, *AlphaEval* identifies failure modes invisible until agents encounter real business requirements; for deploying organizations, it offers ready-made evaluation infrastructure. We open-source the evaluation framework and construction methodology, enabling the community to build production-grounded benchmarks for their own domains—addressing the continuous evolution challenge through collaborative development.

From Capability Measurement to Value Measurement. A distinctive feature of *AlphaEval* is its economic value grounding, which enables a shift from asking "how well does the agent perform?" to "how much value does the agent deliver?" Our domain-weighted analysis reveals that the best agent configuration delivers an estimated $110K–$165K in equivalent professional labor value across the benchmark, while the worst delivers only $70K–$105K. This framing has practical implications: organizations can directly compare agent licensing costs against the value differential between configurations, making agent selection an economic optimization problem rather than a purely technical one.

Limitations. Several limitations should be acknowledged. First, the current benchmark covers six O*NET domains from seven companies; while diverse, it does not yet span all occupational categories where agents are deployed (e.g., legal, education, creative industries). Second, our evaluation relies on a single snapshot in time—both agent capabilities and partner quality standards evolve, and longitudinal tracking remains future work. Third, the economic value estimates, while expert-calibrated, involve assumptions about benefit multipliers and wage distributions that may not generalize across all markets. Fourth, we evaluate only four commercial agent scaffolds; emerging open-source frameworks and custom enterprise pipelines are not yet represented. Finally, the 14 configurations, while covering all models and scaffolds, do not exhaust all possible pairings, and performance on untested combinations may differ.

7 Conclusion

We present *AlphaEval*, a production-grounded benchmark of 94 tasks sourced from seven companies across six O*NET occupational domains, together with a standardized requirement-to-benchmark construction framework. Our evaluation of 14 model–scaffold configurations across six frontier models and four commercial agent products reveals three key insights: (1) the best configuration (Claude Code + Opus 4.6) achieves only 64.41/100, exposing a substantial gap between research benchmark performance and production readiness; (2) scaffold choice matters as much as model choice—the same model can vary by 11–15 points depending on the agent product, confirming that evaluating complete *agent systems* rather than models alone is essential; and (3) domain-weighted economic value reveals that score rankings do not always align with value rankings, providing organizations with a more nuanced basis for agent selection. We identify six production-specific failure modes—cascade dependencies, subjective judgment collapse, information retrieval failures, cross-section inconsistency, constraint misinterpretation, and format compliance—that are invisible to coding-centric benchmarks. By open-sourcing the evaluation framework and construction methodology, we aim to enable community-driven evolution: organizations can adopt our standardized pipeline to build production-grounded benchmarks for their own domains.

Acknowledgments

We thank Keyu Li for his valuable comments, and Tianze Xu and Zhen Huang for their valuable contributions to the early stages of this project.

References

[1] AgentIF Team. 2026. Agentif-oneday: A task-level instruction-following benchmark for general ai agents in daily scenarios. *arXiv preprint arXiv:2601.20613*.

[2] Victor Barres et al. 2025. τ^2-bench: Evaluating conversational agents in a dual-control environment. *arXiv preprint arXiv:2506.07982*.

[3] Jonathan Bragg, Mike D'Arcy, Nishant Balepur, Dan Bareket, Bhavana Dalvi, et al. 2025. Astabench: Rigorous benchmarking of ai agents with a scientific research suite. *arXiv preprint arXiv:2510.21652*.

[4] Keita Broadwater. 2026. Evaluating llm safety under repeated inference via accelerated prompt stress testing. *arXiv preprint arXiv:2602.11786*.

[5] Jialun Cao et al. 2025. Rigor, reliability, and reproducibility matter: A decade-scale survey of 572 code benchmarks. *arXiv preprint arXiv:2501.10711*. Comprehensive guideline with 55-criteria checklist for benchmarks.

[6] Yuan Cao, Dezhi Ran, Mengzhou Wu, Yuzhe Guo, Xin Chen, et al. 2026. Gui-genesis: Automated synthesis of efficient environments with verifiable rewards for gui agent post-training. *arXiv preprint arXiv:2602.14093*.

[7] Arnold Cartagena and Ariane Teixeira. 2026. Mind the gap: Text safety does not transfer to tool-call safety in llm agents. *arXiv preprint arXiv:2602.16943*.

[8] Jun Shern Chan et al. 2024. Mle-bench: Evaluating machine learning agents on machine learning engineering. *arXiv preprint arXiv:2410.07095*.

[9] Chen Chen, Xinlong Hao, Weiwen Liu, Xu Huang, Xingshan Zeng, et al. 2025a. Acebench: Who wins the match point in tool usage? *arXiv preprint arXiv:2501.12851*.

[10] Haorui Chen, Chengze Li, and Jia Li. 2025b. Featbench: Towards more realistic evaluation of feature-level code generation. *arXiv preprint arXiv:2509.22237*.

[11] Tingting Chen, Srinivas Anumasa, Beibei Lin, Vedant Shah, Anirudh Goyal, and Dianbo Liu. 2025c. Autobench: An automated benchmark for scientific discovery in llms. *arXiv preprint arXiv:2502.15224*.

[12] François Chollet and ARC Prize Team. 2025. Arc-agi-2: A new challenge for frontier ai reasoning systems. *arXiv preprint arXiv:2505.11831*.

[13] Yun-Shiuan Chuang, Chaitanya Kulkarni, Alec Chiu, Avinash Thangali, Zijie Pan, et al. 2026. Toward scalable verifiable reward: Proxy state-based evaluation for multi-turn tool-calling llm agents. *arXiv preprint arXiv:2602.16246*.

[14] Pepijn Cobben, Xuanqiang Angelo Huang, Thao Amelia Pham, Isabel Dahlgren, Terry Jingchen Zhang, and Zhijing Jin. 2026. Gt-harmbench: Benchmarking ai safety risks through the lens of game theory. *arXiv preprint arXiv:2602.12316*.

[15] Cybergod AGI Research. 2025. Vimgolf-gym: Openai gym style vimgolf environment and benchmark.

[16] Mariya Davydova, Daniel Jeffries, Patrick Barker, Arturo Márquez Flores, and Sinéad Ryan. 2025. Osuniverse: Benchmark for multimodal gui-navigation ai agents. *arXiv preprint arXiv:2505.03570*.

[17] Titouan Duston, Shuo Xin, Yang Sun, Daoguang Zan, Aoyan Li, et al. 2025. Ainsteinbench: Benchmarking coding agents on scientific repositories. *arXiv preprint arXiv:2512.21373*.

[18] Yukang Feng, Jianwen Sun, Zelai Yang, Jiaxin Ai, Chuanhao Li, et al. 2026. Longcli-bench: A preliminary benchmark and study for long-horizon agentic programming in command-line interfaces. *arXiv preprint arXiv:2602.14337*.

[19] Romain Froger, Pierre Andrews, Matteo Bettini, Amar Budhiraja, Ricardo Silveira Cabral, et al. 2026. Gaia2: Benchmarking llm agents on dynamic and asynchronous environments. *arXiv preprint arXiv:2602.11964*.

[20] Chaoyou Fu, Yuhan Dai, Yongdong Luo, Lei Li, Shuhuai Ren, et al. 2024. Video-mme: The first-ever comprehensive evaluation benchmark of multi-modal llms in video analysis. *arXiv preprint arXiv:2405.21075*.

[21] Aniketh Garikaparthi, Manasi Patwardhan, and Arman Cohan. 2026. Researchgym: Evaluating language model agents on real-world ai research. *arXiv preprint arXiv:2602.15112*.

[22] Paul Gauthier. 2024. Aider polyglot benchmark.

[23] Elliot Glazer, Ege Erdil, Tamay Besiroglu, Diego Chicharro, Evan Chen, et al. 2024. Frontiermath: A benchmark for evaluating advanced mathematical reasoning in ai. *arXiv preprint arXiv:2411.04872*.

[24] Thibaud Gloaguen, Niels Mündler, Mark Müller, Veselin Raychev, and Martin Vechev. 2026. Evaluating agents.md: Are repository-level context files helpful for coding agents? *arXiv preprint arXiv:2602.11988*.

[25] Harvard-MIT. 2025. Harvard-mit mathematics tournament 2025. https://www.hmmt.org/.

[26] Zexue He, Yu Wang, Churan Zhi, Yuanzhe Hu, Tzu-Ping Chen, et al. 2026. Memoryarena: Benchmarking agent memory in interdependent multi-session agentic tasks. *arXiv preprint arXiv:2602.16313*.

[27] Dan Hendrycks, Collin Burns, Steven Basart, Andy Zou, Mantas Mazeika, Dawn Song, and Jacob Steinhardt. 2020. Measuring massive multitask language understanding. *arXiv preprint arXiv:2009.03300*.

[28] Lingxiang Hu, Yiding Sun, Tianle Xia, Wenwei Li, Ming Xu, et al. 2026. Ad-bench: A real-world, trajectory-aware advertising analytics benchmark for llm agents. *arXiv preprint arXiv:2602.14257*.

[29] Tianyu Hua, Harper Hua, Violet Xiang, Benjamin Klieger, Sang T. Truong, et al. 2025. Researchcodebench: Benchmarking llms on implementing novel machine learning research code. *arXiv preprint arXiv:2506.02314*.

[30] Naman Jain, King Han, Alex Gu, et al. 2024. Livecodebench: Holistic and contamination free evaluation of llms for code. *arXiv preprint arXiv:2403.07974*.

[31] JetBrains. 2025. Dpai arena.

[32] Tanqiu Jiang, Yuhui Wang, Jiacheng Liang, and Ting Wang. 2026. Agentlab: Benchmarking llm agents against long-horizon attacks. *arXiv preprint arXiv:2602.16901*.

[33] Carlos E Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. 2024. Swe-bench: Can language models resolve real-world github issues? *arXiv preprint arXiv:2310.06770*.

[34] Liqiang Jing et al. 2024. Dsbench: How far are data science agents from becoming data science experts? *arXiv preprint arXiv:2409.07703*.

[35] Serin Kim, Sangam Lee, and Dongha Lee. 2026a. Persona2web: Benchmarking personalized web agents for contextual reasoning with user history. *arXiv preprint arXiv:2602.17003*.

[36] Sunghwan Kim, Ryang Heo, Yongsik Seo, Jinyoung Yeo, and Dongha Lee. 2026b. Agenticshop: Benchmarking agentic product curation for personalized web shopping. *arXiv preprint arXiv:2602.12315*.

[37] Patrick Tser Jern Kon, Jiachen Liu, Xinyi Zhu, Qiuyi Ding, Jingjia Peng, et al. 2025. Exp-bench: Can ai conduct ai research experiments? *arXiv preprint arXiv:2505.24785*.

[38] Ziyan Kuang et al. 2025. From scores to skills: A cognitive diagnosis framework for evaluating financial large language models. *arXiv preprint arXiv:2508.13491*. Knowledge-skill level evaluation for financial LLMs.

[39] Fangyu Lei, Jinxiang Meng, Yiming Huang, Junjie Zhao, Yitong Zhang, et al. 2025. Dacomp: Benchmarking data agents across the full data intelligence lifecycle. *arXiv preprint arXiv:2512.04324*.

[40] Letta. 2025a. Context-bench: A benchmark for agentic context engineering. https://www.sundeepteki.org/blog/context-bench.

[41] Letta. 2025b. Letta evals: Evaluating agents that learn. https://www.letta.com/blog/letta-evals.

[42] Bowen Li et al. 2024. Prompting large language models to tackle the full software development lifecycle: A case study. *arXiv preprint arXiv:2403.08604*.

[43] Eric Li and Jim Waldo. 2024. Websuite: Systematically evaluating why web agents fail. *arXiv preprint arXiv:2406.01623*.

[44] Junlong Li, Wenshuo Zhao, Jian Zhao, Weihao Zeng, Haoze Wu, et al. 2025. The tool decathlon: Benchmarking language agents for diverse, realistic, and long-horizon task execution. *arXiv preprint arXiv:2510.25726*.

[45] Keyu Li, Junhao Shi, Yang Xiao, Mohan Jiang, et al. 2026a. Agencybench: Benchmarking the frontiers of autonomous agents in 1m-token real-world contexts. *arXiv preprint arXiv:2601.11044*.

[46] Xiangyi Li, Wenbo Chen, Yimin Liu, Shenghan Zheng, Xiaokun Chen, et al. 2026b. Skillsbench: Benchmarking how well agent skills work across diverse tasks. *arXiv preprint arXiv:2602.12670*.

[47] Xu Li, Simon Yu, Minzhou Pan, Yiyou Sun, Bo Li, et al. 2026c. Unsafer in many turns: Benchmarking and defending multi-turn safety risks in tool-using agents. *arXiv preprint arXiv:2602.13379*.

[48] Kaiqu Liang, Julia Kruk, Shengyi Qian, Xianjun Yang, Shengjie Bi, et al. 2026. Learning personalized agents from human feedback. *arXiv preprint arXiv:2602.16173*.

[49] Ling Lin, Yang Bai, Heng Su, Congcong Zhu, Yaoxing Wang, et al. 2026. Oodbench: Out-of-distribution benchmark for large vision-language models. *arXiv preprint arXiv:2602.18094*.

[50] Xiao Liu, Hao Yu, Hanchen Zhang, Yifan Xu, Xuanyu Lei, Hanyu Lai, Yu Gu, Hangliang Ding, Kaiwen Men, Kejuan Yang, et al. 2023. Agentbench: Evaluating llms as agents. *arXiv preprint arXiv:2308.03688*. Published at ICLR 2024.

[51] Lajanugen Logeswaran, Jaekyeom Kim, Sungryull Sohn, Creighton Glasscock, and Honglak Lee. 2026. Scaling web agent training through automatic data generation and fine-grained evaluation. *arXiv preprint arXiv:2602.12544*.

[52] Pan Lu, Hritik Bansal, Tony Xia, Jiacheng Liu, Chunyuan Li, Hannaneh Hajishirzi, Hao Cheng, Kai-Wei Chang, Michel Galley, and Jianfeng Gao. 2024. Mathvista: Evaluating mathematical reasoning of foundation models in visual contexts. *arXiv preprint arXiv:2310.02255*.

[53] Pengrui Lu, Shiqi Zhang, Yunzhong Hou, Lyumanshan Ye, Chaoyi Huang, et al. 2026. Projdevbench: Benchmarking ai coding agents on end-to-end project development. *arXiv preprint arXiv:2602.01655*.

[54] Ziyang Luo, Zhiqi Shen, Wenzhuo Yang, Zirui Zhao, Prathyusha Jwalapuram, Amrita Saha, Doyen Sahoo, Silvio Savarese, Caiming Xiong, and Junnan Li. 2025. Mcp-universe: Benchmarking large language models with real-world model context protocol servers. *arXiv preprint arXiv:2508.14704*. 6 domains, 11 MCP servers, 231 tasks.

[55] Chang Ma, Junlei Zhang, Zhihao Zhu, Cheng Yang, Yujiu Yang, Yaohui Jin, Zhenzhong Lan, Lingpeng Kong, and Junxian He. 2024. Agentboard: An analytical evaluation board of multi-turn llm agents.

[56] MAA. 2025. American invitational mathematics examination (aime) 2025. https://www.kaggle.com/benchmarks/open-benchmarks/aime-2025.

[57] Lingjun Mao, Jiawei Ren, Kun Zhou, Jixuan Chen, Ziqiao Ma, and Lianhui Qin. 2025. Deliverybench: Can agents earn profit in real world? *arXiv preprint arXiv:2512.19234*.

[58] Mike A. Merrill, Alexander G. Shaw, Nicholas Carlini, Boxuan Li, Harsh Raj, Ivan Bercovich, Lin Shi, Jeong Yeon Shin, Thomas Walshe, E. Kelly Buchanan, Junhong Shen, Guanghao Ye, Haowei Lin, Jason Poulos, Maoyu Wang, Marianna Nezhurina, Jenia Jitsev, Di Lu, Orfeas Menis Mastromichalakis, Zhiwei Xu, Zizhao Chen, Yue Liu, Robert Zhang, Leon Liangyu Chen, Anurag Kashyap, Jan-Lucas Uslu, Jeffrey Li, Jianbo Wu, Minghao Yan, Song Bian, Vedang Sharma, Ke Sun, Steven Dillmann, Akshay Anand, Andrew Lanpouthakoun, Bardia Koopah, Changran Hu, Etash Guha, Gabriel H. S. Dreiman, Jiacheng Zhu, Karl Krauth, Li Zhong, Niklas Muennighoff, Robert Amanfu, Shangyin Tan, Shreyas Pimpalgaonkar, Tushar Aggarwal, Xiangning Lin, Xin Lan, Xuandong Zhao, Yiqing Liang, Yuanli Wang, Zilong Wang, Changzhi Zhou, David Heineman, Hange Liu, Harsh Trivedi, John Yang, Junhong Lin, Manish Shetty, Michael Yang, Nabil Omi, Negin Raoof, Shanda Li, Terry Yue Zhuo, Wuwei Lin, Yiwei Dai, Yuxin Wang, Wenhao Chai, Shang Zhou, Dariush Wahdany, Ziyu She, Jiaming Hu, Zhikang Dong, Yuxuan Zhu, Sasha Cui, Ahson Saiyed, Arinbjörn Kolbeinsson, Jesse Hu, Christopher Michael Rytting, Ryan Marten, Yixin Wang, Alex Dimakis, Andy Konwinski, and Ludwig Schmidt. 2026. Terminal-bench: Benchmarking agents on hard, realistic tasks in command line interfaces.

[59] Grégoire Mialon, Clémentine Fourrier, Craig Swift, Thomas Wolf, Yann LeCun, and Thomas Scialom. 2023. Gaia: A benchmark for general ai assistants. *arXiv preprint arXiv:2311.12983*.

[60] Rui Min, Zile Qiao, Ze Xu, Jiawen Zhai, Wenyu Gao, Xuanzhong Chen, Haozhen Sun, Zhen Zhang, Xinyu Wang, Hong Zhou, Wenbiao Yin, Bo Zhang, Xuan Zhou, Ming Yan, Yong Jiang, Haicheng Liu, Liang Ding, Ling Zou, Yi R. Fung, Yalong Li, and Pengjun Xie. 2025. Ecombench: Towards holistic evaluation of foundation agents in e-commerce. *arXiv preprint arXiv:2512.08868*. 7 categories, quarterly updates, human-in-the-loop framework.

[61] Samuel Miserendino, Michele Wang, Tejal Patwardhan, and Johannes Heidecke. 2025. Swe-lancer: Can frontier llms earn $1 million from real-world freelance software engineering? *arXiv preprint arXiv:2502.12115*.

[62] Niels Mündler, Mark Niklas Mueller, Jingxuan He, and Martin Vechev. 2024. SWT-bench: Testing and validating real-world bug-fixes with code agents. In *Advances in Neural Information Processing Systems*.

[63] OpenAI. 2024. Mmmlu: Massive multilingual multitask language understanding. *HuggingFace Dataset*.

[64] OpenAI. 2025. Openai-mrcr: Multi-round coreference resolution.

[65] Krista Opsahl-Ong, Arnav Singhvi, Jasmine Collins, Ivan Zhou, Cindy Wang, Ashutosh Baheti, Owen Oertell, Jacob Portes, Sam Havens, Erich Elsen, Michael Bendersky, Matei Zaharia, and Xing Chen. 2026. Officeqa pro: An enterprise benchmark for end-to-end grounded reasoning.

[66] Anne Ouyang, Simon Guo, Simran Arora, Alex L. Zhang, William Hu, Christopher Ré, and Azalia Mirhoseini. 2025. Kernelbench: Can llms write efficient gpu kernels?

[67] Melissa Z. Pan et al. 2025. Measuring agents in production. *arXiv preprint arXiv:2512.04123*. Survey of 306 practitioners with 20 in-depth case studies.

[68] Arjun Panickssery, Samuel R Bowman, and Shi Feng. 2024. Llm evaluators recognize and favor their own generations. *arXiv preprint arXiv:2404.13076*. Published at NeurIPS 2024.

[69] Shishir G. Patil, Huanzhi Mao, Charlie Cheng-Jie Ji, Fanjia Yan, Vishnu Suresh, Ion Stoica, and Joseph E. Gonzalez. 2025. The berkeley function calling leaderboard (bfcl): From tool use to agentic evaluation of large language models. In *Forty-second International Conference on Machine Learning*.

[70] Ivo Petrov, Jasper Dekoninck, Lyuben Baltadzhiev, Maria Drencheva, Kristian Minchev, et al. 2025. Proof or bluff? evaluating llms on 2025 usa math olympiad. *arXiv preprint arXiv:2503.21934*.

[71] Long Phan, Alice Gatti, Ziwen Han, Nathaniel Li, Josephina Hu, et al. 2025. Humanity's last exam. *arXiv preprint arXiv:2501.14249*.

[72] Shuofei Qiao, Runnan Fang, Zhisong Qiu, Xiaobin Wang, Ningyu Zhang, et al. 2024. Benchmarking agentic workflow generation. *arXiv preprint arXiv:2410.07869*.

[73] Shanghaoran Quan, Jiaxi Yang, Bowen Yu, Bo Zheng, Dayiheng Liu, et al. 2025. Codeelo: Benchmarking competition-level code generation of llms with human-comparable elo ratings. *arXiv preprint arXiv:2501.01257*.

[74] David Rein, Betty Li Hou, Asa Cooper Stickland, Jackson Petty, Richard Yuanzhe Pang, Julien Dirani, Julian Michael, and Samuel R Bowman. 2023. Gpqa: A graduate-level google-proof q&a benchmark. *arXiv preprint arXiv:2311.12022*.

[75] Gabriel Roccabruna, Olha Khomyn, and Giuseppe Riccardi. 2026. Mateo: A multimodal benchmark for temporal reasoning and planning in lvlms. *arXiv preprint arXiv:2602.14589*.

[76] Yujiong Shen, Yajie Yang, Zhiheng Xi, Binze Hu, Huayu Sha, et al. 2026. Sciagentgym: Benchmarking multi-step scientific tool-use in llm agents. *arXiv preprint arXiv:2602.12984*.

[77] Zachary S. Siegel, Sayash Kapoor, Nitya Nagdir, Benedikt Stroebl, and Arvind Narayanan. 2024. Core-bench: Fostering the credibility of published research through a computational reproducibility agent benchmark. *arXiv preprint arXiv:2409.11363*.

[78] Spring Community. 2025. Spring ai bench.

[79] Giulio Starace, Oliver Jaffe, Dane Sherburn, James Aung, Jun Shern Chan, et al. 2025. Paperbench: Evaluating ai's ability to replicate ai research. *arXiv preprint arXiv:2504.01848*.

[80] Jiazheng Sun, Mingxuan Li, Yingying Zhang, Jiayang Niu, Yachen Wu, et al. 2026. Ambibench: Benchmarking mobile gui agents beyond one-shot instructions in the wild. *arXiv preprint arXiv:2602.11750*.

[81] Nivya Talokar, Ayush K Tarun, Murari Mandal, Maksym Andriushchenko, and Antoine Bosselut. 2026. Helpful to a fault: Measuring illicit assistance in multi-turn, multilingual llm agents. *arXiv preprint arXiv:2602.16346*.

[82] Haibo Tong, Feifei Zhao, et al. 2026. Foresightsafety bench: A frontier risk evaluation and governance framework towards safe ai. *arXiv preprint arXiv:2602.14135*.

[83] Harsh Trivedi, Tushar Khot, Mareike Hartmann, Ruskin Manku, Vinty Dong, et al. 2024. Appworld: A controllable world of apps and people for benchmarking interactive coding agents. *arXiv preprint arXiv:2407.18901*.

[84] U.S. Department of Labor. 2025. O*NET OnLine. Accessed: 2026-03-01.

[85] Alisa Vinogradova, Vlad Vinogradov, Luba Greenwood, Ilya Yasny, Dmitry Kobyzev, et al. 2026. Hunt globally: Wide search ai agents for drug asset scouting in investing, business development, and competitive intelligence. *arXiv preprint arXiv:2602.15019*.

[86] Jun Wang, Jiamu Zhou, Muning Wen, Xiaoyun Mo, Haoyu Zhang, et al. 2024. Hammerbench: Fine-grained function-calling evaluation in real mobile device scenarios. *arXiv preprint arXiv:2412.16516*.

[87] Shirui Wang et al. 2025a. A novel evaluation benchmark for medical llms: Illuminating safety and effectiveness in clinical domains. *arXiv preprint arXiv:2507.23486*. 30 criteria: 17 safety + 13 effectiveness metrics.

[88] Weixuan Wang, Dongge Han, Daniel Madrigal Diaz, Jin Xu, Victor Rühle, and Saravan Rajmohan. 2025b. Odysseybench: Evaluating llm agents on long-horizon complex office application workflows. *arXiv preprint arXiv:2508.09124*.

[89] Zora Zhiruo Wang, Sanidhya Vijayvargiya, Aspen Chen, Hanmo Zhang, Venu Arvind Arangarajan, Jett Chen, Valerie Chen, Diyi Yang, Daniel Fried, and Graham Neubig. 2026. How well does agent development reflect real-world work? *arXiv preprint arXiv:2603.01203*.

[90] Jason Wei, Zhiqing Sun, Spencer Papay, Scott McKinney, Jeffrey Han, et al. 2025. Browsecomp: A simple yet challenging benchmark for browsing agents. *arXiv preprint arXiv:2504.12516*.

[91] Richard Willis, Jianing Zhao, Yali Du, and Joel Z. Leibo. 2026. Evaluating collective behaviour of hundreds of llm agents. *arXiv preprint arXiv:2602.16662*.

[92] xbench Team. 2025. xbench: Tracking agents productivity scaling with profession-aligned real-world evaluations. *arXiv preprint arXiv:2506.13651*.

[93] Zikai Xiao, Jianhong Tu, Chuhang Zou, Yuxin Zuo, Zhi Li, et al. 2026. Webworld: A large-scale world model for web agent training. *arXiv preprint arXiv:2602.14721*.

[94] Tianbao Xie, Danyang Zhang, Jixuan Chen, Xiaochuan Li, Siheng Zhao, et al. 2024. Osworld: Benchmarking multimodal agents for open-ended tasks in real computer environments. *arXiv preprint arXiv:2404.07972*.

[95] Frank F Xu et al. 2024. Theagentcompany: Benchmarking llm agents on consequential real world tasks. *arXiv preprint arXiv:2412.14161*.

[96] John Yang, Carlos E Jimenez, et al. 2024. Swe-bench multimodal: Do ai systems generalize to visual software domains? *arXiv preprint arXiv:2410.03859*.

[97] Qianyu Yang, Yang Liu, Jiaqi Li, Jun Bai, Hao Chen, Kaiyuan Chen, Tiliang Duan, Jiayun Dong, Xiaobo Hu, Zixia Jia, Yang Liu, Tao Peng, Yixin Ren, Ran Tian, Zaiyuan Wang, Yanglihong Xiao, Gang Yao, Lingyue Yin, Ge Zhang, Chun Zhang, Jianpeng Jiao, Zilong Zheng, and Yuan Gong. 2026. $onemillion-bench: How far are language agents from human experts? *arXiv preprint arXiv:2603.07980*.

[98] Shunyu Yao, Noah Shinn, Pedram Razavi, and Karthik Narasimhan. 2024. τ-bench: A benchmark for tool-agent-user interaction in real-world domains. *arXiv preprint arXiv:2406.12045*.

[99] Xiang Yue, Yuansheng Ni, Kai Zhang, Tianyu Zheng, et al. 2023. Mmmu: A massive multi-discipline multimodal understanding and reasoning benchmark for expert agi. *arXiv preprint arXiv:2311.16502*.

[100] Daoguang Zan, Zhirong Huang, Wei Liu, Hanwu Chen, Linhao Zhang, et al. 2025. Multi-swe-bench: A multilingual benchmark for issue resolving. *arXiv preprint arXiv:2504.02605*.

[101] Andy Zhang et al. 2024. Cybench: A framework for evaluating cybersecurity capabilities and risks of language models. *arXiv preprint arXiv:2408.08926*.

[102] Andy K. Zhang, Joey Ji, Celeste Menders, Riya Dulepet, Thomas Qin, Ron Y. Wang, Junrong Wu, Kyleen Liao, Jiliang Li, Jinghan Hu, Sara Hong, Nardos Demilew, Shivatmica Murgai, Jason Tran, Nishka Kacheria, Ethan Ho, Denis Liu, Lauren McLane, Olivia Bruvik, Dai-Rong Han, Seungwoo Kim, Akhil Vyas, Cuiyuanxiu Chen, Ryan Li, Weiran Xu, Jonathan Z. Ye, Prerit Choudhary, Siddharth M. Bhatia, Vikram Sivashankar, Yuxuan Bao, Dawn Song, Dan Boneh, Daniel E. Ho, and Percy Liang. 2025. Bountybench: Dollar impact of ai agent attackers and defenders on real-world cybersecurity systems. *arXiv preprint arXiv:2505.15216*.

[103] Huanyao Zhang, Jiepeng Zhou, Bo Li, Bowen Zhou, Yanzhe Shan, et al. 2026. Browsecomp-v^3: A visual, vertical, and verifiable benchmark for multimodal browsing agents. *arXiv preprint arXiv:2602.12876*.

[104] Lianmin Zheng, Wei-Lin Chiang, Ying Sheng, Siyuan Zhuang, Zhanghao Wu, Yonghao Zhuang, Zi Lin, Zhuohan Li, Dacheng Li, Eric P Xing, et al. 2023. Judging llm-as-a-judge with mt-bench and chatbot arena. *arXiv preprint arXiv:2306.05685*. Published at NeurIPS 2023.

[105] Shuyan Zhou, Frank F Xu, Hao Zhu, Xuhui Zhou, Robert Lo, Abishek Sridhar, Xianyi Cheng, et al. 2024. Webarena: A realistic web environment for building autonomous agents. *arXiv preprint arXiv:2307.13854*.

[106] Mingchen Zhuge, Changsheng Zhao, Dylan Ashley, Wenyi Wang, Dmitrii Khizbullin, Yunyang Xiong, Zechun Liu, Ernie Chang, Raghuraman Krishnamoorthi, Yuandong Tian, Yangyang Shi, Vikas Chandra, and Jürgen Schmidhuber. 2024. Agent-as-a-judge: Evaluate agents with agents. *arXiv preprint arXiv:2410.10934*.

A Evaluation Infrastructure Details

The evaluation pipeline proceeds in six stages: (1) **Task Loading**: the framework scans the benchmark directory and constructs a task queue filterable by domain, difficulty, or evaluation type; (2) **Environment Provisioning**: Docker containers are instantiated with task input files; (3) **Agent Invocation**: the agent receives `query.md` and interacts through a standardized interface; (4) **Output Collection**: artifacts are collected from the results directory; (5) **Evaluation Dispatch**: outputs are routed to the appropriate evaluator; (6) **Result Aggregation**: results are aggregated into domain-level and benchmark-level summaries.

All task inputs, evaluation scripts, and reference answers are version-controlled. Docker images and agent scaffold versions are pinned for reproducibility (Claude Code 2.1.70, Codex 0.80.0/0.111.0, GitHub Copilot 1.0.10, Cursor 2026.03.11; see Table 6 for details). Each run generates structured logs with full interaction traces, raw outputs, and scores in JSON format.

B Scoring and Aggregation Methodology

Per-Task Scoring. Each task's evaluation rubric script (`rubric.py`) outputs a standardized score $s \in [0, 1]$, which is scaled to $[0, 100]$ for reporting. For tasks composing multiple evaluation paradigms, the rubric script internally handles the composition—typically as a weighted average of sub-evaluations, where weights are co-designed with domain experts during the task formalization stage. The final per-task score thus captures multi-dimensional quality in a single number:

$$s_{\text{task}} = \sum_{k=1}^{K} w_k \cdot e_k(a, t), \quad \text{where} \sum_{k=1}^{K} w_k = 1 \tag{1}$$

where $e_k(a, t) \in [0, 1]$ is the evaluation result from paradigm k for agent a on task t, and w_k is the expert-assigned weight for paradigm k.

Domain-Level Aggregation. Domain scores are computed as unweighted arithmetic means across all tasks within a domain:

$$S_{\text{domain}}(a) = \frac{1}{|T_d|} \sum_{t \in T_d} s_{\text{task}}(a, t) \times 100 \tag{2}$$

where T_d is the set of tasks in domain d.

Overall Aggregation. The overall benchmark score is the unweighted arithmetic mean across the six domain scores, giving each domain equal weight regardless of task count:

$$S_{\text{overall}}(a) = \frac{1}{6} \sum_{d=1}^{6} S_{\text{domain}}(a) \tag{3}$$

This equal-weighting ensures that domains with fewer tasks (e.g., Human Resources with 11 tasks) receive the same influence as domains with more tasks (e.g., Procurement & Operations with 23 tasks), reflecting the principle that production readiness requires competence across all occupational domains.

C Agent System Configuration

All agent systems are invoked via their respective command-line interfaces (CLIs) within Docker-sandboxed environments. Table 6 summarizes the configuration details.

Agent Product	Version	Interface	Models
Claude Code	2.1.70	`claude` CLI	All 6 models
Codex	0.111.0	`codex` CLI	GPT-5.2, Opus 4.6
Codex	0.80.0	`codex` CLI	GLM-5, Kimi K2.5
GitHub Copilot	1.0.10	`copilot-cli`	GPT-5.2, Gemini 3 Pro, Opus 4.6
Cursor	2026.03.11	`cursor` CLI	Opus 4.6

Table 6: Agent system configurations with pinned versions. Codex version varies by model due to API compatibility requirements.

Trajectory Recording. Each evaluation run produces a complete execution trajectory capturing: (1) all tool calls and their arguments, (2) intermediate reasoning steps (where exposed by the scaffold), (3) file read/write operations, and (4) the final output artifacts. These trajectories enable post-hoc failure analysis—for instance, identifying whether a low score resulted from the model failing to find relevant information (search failure) or finding it but incorporating it incorrectly (reasoning failure). Although some agent products are closed-source, the CLI interface ensures that the agent's observable behavior—tool calls, outputs, and timing—is fully recorded and reproducible.

Model Configuration. All models are evaluated using their default API configurations (temperature, max tokens, etc.) as provided by each scaffold. We do not modify model parameters, as the goal is to evaluate agent *products* as deployed, not to optimize individual model settings. This mirrors how production users interact with these systems.

D Evaluation Taxonomy Coverage

Table 7 maps the leaf-node evaluation types from our taxonomy (Figure 2) to the domains that employ them. *AlphaEval* covers 8 of the 14 leaf-node types across four paradigms; the uncovered types (word-embedding similarity, repo integration testing, auto-generated rubrics, human-AI collaborative rubrics, and pairwise ranking) are either inapplicable to production tasks or reserved for future expansion.

Paradigm	Leaf-node Type	Covered	Tasks	Domains
Reference Verif.	Fuzzy matching	✓	42	HR, F&I, P&O
	Exact matching	✓	33	HR, F&I, P&O
	Word-embed. sim.	—	0	—
	LLM semantic	✓	49	F&I, H&LS, TR
Test Case Verif.	Code unit testing	✓	20	P&O
	Repo integ. test	—	0	—
	Env. state verif.	✓	13	F&I, SE
Formal Verif.	Math proof	✓	25	H&LS, F&I, P&O
	Code/logic	✓	2	H&LS
Rubric Assess.	Human-authored	✓	49	F&I, H&LS, TR
	Auto-generated	—	0	—
	Human-AI collab.	—	0	—
Competition	Ranking	—	0	—

Table 7: Fine-grained evaluation type coverage across the paradigms implemented by *AlphaEval*.

E Benchmark Statistics

Table 8 summarizes the key statistics of *AlphaEval*.

Statistic	Value
Total tasks	94
Domains covered	6
Partner companies	7
Input modality distribution	
PDF-primary	∼42% (39 tasks)
Excel/CSV-primary	∼21% (20 tasks)
Markdown/Text	∼25% (24 tasks)
Code/YAML	∼12% (11 tasks)
Avg. execution time	14 minutes
Avg. interaction turns	46

Table 8: Benchmark statistics of *AlphaEval*. Execution time and turns measured on Claude Code + Opus 4.6.

The input modality distribution reflects the heterogeneity of production work: PDF-primary tasks (42%) dominate due to the prevalence of business documents in finance, healthcare, and HR, while structured data (21%) and markdown/text (25%) cover procurement and technology research domains respectively. The average execution time of 14 minutes and 46 interaction turns per task underscore the long-horizon nature of production tasks compared to typical research benchmarks.

F Multi-label Evaluation Composition

A key design principle of *AlphaEval* is that individual tasks compose multiple evaluation types rather than relying on a single metric. Table 9 details this composition by domain. Procurement & Operations has the highest average (3.8 types per task), reflecting the need to verify constraint satisfaction, cost optimality, and output format simultaneously. Human Resources has the lowest (2.0), as resume screening primarily relies on matching-based evaluation. Every task in the benchmark employs at least two evaluation types, with a benchmark-wide average of 2.8.

Domain	Tasks	Avg.	Types Composed
Human Resources	11	2.0	Fuzzy + Exact match
Finance & Investment	22	2.5	Rubric + LLM semantic + Structural verif. + Match
Procurement & Operations	23	3.8	Fuzzy + Exact + Unit test + Math + LLM
Software Engineering	11	2.5	Env. state verif. + Functional verif. + LLM
Healthcare & Life Sci.	16	2.8	Rubric + LLM + Math + Code + Match
Technology Research	11	2.5	Rubric + LLM semantic + Factual verif.
Overall	**94**	**2.8**	**8 types, 100% $\geq$2**

Table 9: Multi-label evaluation composition by domain. Each domain composes multiple evaluation types within individual task rubrics.

G Economic Value Estimation Methodology

We annotate each task with its human replacement cost through a two-stage pipeline: automated AI estimation followed by domain expert calibration.

G.1 Stage 1: AI Estimation

For each task, an LLM analyzes the task specification (`query.md`) and produces: (1) a list of required professional roles, (2) estimated hours per role, and (3) a complexity rating. The core estimation principle is: "estimate the time a qualified human professional would need to complete *this specific task*"—not the cost of building a replacement system.

Wage Data Sources. U.S. wages are queried from the Bureau of Labor Statistics (BLS) Occupational Employment and Wage Statistics via SOC codes. Chinese wages are sourced from publicly available salary data for Beijing-based positions. Each role maps to a standardized SOC code (e.g., Software Developer $\rightarrow$ 15-1252, Recruiter $\rightarrow$ 13-1071, Data Analyst $\rightarrow$ 15-2051).

Cost Calculation.

$$\text{Value} = \text{Hourly Rate} \times \text{Benefit Multiplier} \times \text{Hours} \tag{4}$$

where the benefit multiplier is 1.3$\times$ for U.S. (healthcare, retirement, PTO), derived from BLS Employer Costs for Employee Compensation data, following the methodology of Yang et al. (2026), and 1.45$\times$ for China (statutory "Five Insurances and One Fund" contributions plus annualized bonuses), based on Beijing municipal social insurance contribution rates. Hourly rates use the 25th–75th percentile range to produce value intervals.

G.2 Stage 2: Expert Calibration

Domain practitioners review AI estimates and apply correction methods tailored to each domain:

- **Individualized assessment** (Software Engineering, factor 0.38): Experts evaluated each task considering code reuse across tasks, reducing total hours from 2,458 to 946. The largest reduction (78%) was for the task marketplace app where transaction components were reusable.
- **Uniform scaling** (Finance & Investment, factor 0.50): Expert judged AI systematically overestimated analytical tasks; all hours halved.
- **Quantitative standardization** (Human Resources, factor 0.64): Expert provided an empirical rate (20 resumes = 1 hour) to recalculate screening tasks.

Salary data from https://www.salaryexpert.com/, cross-referenced with local market surveys.
https://www.bls.gov/news.release/ecec.nr0.htm
https://rsj.beijing.gov.cn/

- **Fixed-rate override** (Procurement & Operations, factor 0.33): Expert specified domain hourly rates (¥213/hr), implying 67% hour reduction.
- **Selective adjustment** (Finance & Investment, factor 1.04): Expert increased two complex tasks to 150% of baseline; others unchanged.
- **Module re-estimation** (Healthcare & Life Sciences, factor 1.54): Expert reorganized 10 tasks into 3 functional modules, *increasing* hours from 86 to 132, reflecting AI's underestimation of clinical protocol complexity.
- **No adjustment** (Finance & Investment, factor 1.00): Expert confirmed AI estimates as reasonable.
- **Uniform scaling** (Technology Research, factor 0.80): Expert judged AI estimates slightly high; all hours reduced by 20%, from 298 to 238 hours.

Key Observations. All six domains have completed expert calibration. The correction factors range from 0.33 to 1.54, with no consistent direction: AI overestimates routine tasks (procurement, software engineering) but underestimates domain-specialized tasks (clinical research). This validates the two-stage approach—neither pure AI estimation nor pure expert estimation alone would suffice. Table 10 summarizes the per-domain economic value after expert calibration.

Table 10: Economic value summary by domain after expert calibration. Hours and values reflect expert-adjusted estimates.

Domain	Tasks	Hours	USD (K)	CNY (K)
Human Resources	11	39	1.8–2.7	4.1–6.4
Finance & Investment	22	803	49.6–74.4	165.3–248.7
Procurement & Operations	23	236	20.4–30.6	49.1–49.7
Software Engineering	11	946	61.2–91.8	116.9–179.9
Healthcare & Life Sci.	16	154	6.7–9.5	19.4–28.0
Technology Research	11	242	14.6–21.8	36.3–57.7
Total	**94**	**2,420**	**154–231**	**391–570**

G.3 Sensitivity Analysis

To assess the robustness of the economic value estimates, we conduct a sensitivity analysis varying the three key parameters: benefit multipliers, hourly rate percentiles, and expert correction factors.

Benefit Multiplier Sensitivity. Varying the U.S. multiplier from 1.2 to 1.4 (baseline: 1.3) and the China multiplier from 1.3 to 1.6 (baseline: 1.45) produces total value ranges of \$141K–\$253K (USD) and ¥358K–¥627K (CNY), representing $\pm 9\%$ variation from the baseline estimates. The estimates are most sensitive to the China multiplier due to the larger proportion of China-based tasks.

Correction Factor Sensitivity. To test the impact of expert calibration, we compute valuations under three scenarios: (1) AI-only estimation (no expert correction), (2) baseline expert-calibrated values, and (3) uniform 20% increase in all correction factors. Results: AI-only yields 3,240 hours (\$217K–\$326K), baseline yields 2,420 hours (\$154K–\$231K), and the inflated scenario yields 2,904 hours (\$185K–\$277K). Expert calibration reduces the AI estimate by 25%, primarily driven by the Software Engineering domain where AI overestimated development hours by $2.6\times$.

Hourly Rate Sensitivity. Shifting from the 25th–75th percentile range to the 10th–90th percentile range widens the value interval to \$128K–\$289K (a 61% wider band). However, the median estimate (\$192K) remains stable, indicating that the central tendency is robust even under wider wage assumptions.

These analyses confirm that while the absolute dollar values are sensitive to parameter choices (particularly the China benefit multiplier and expert correction factors), the relative ordering of domains by economic value and the overall conclusion that *AlphaEval* captures economically meaningful work remain stable across all tested scenarios.

H Detailed Error Analysis

We conduct a systematic error analysis across $\sim$130 agent$\times$model evaluation results, categorizing failures by their cognitive root causes rather than surface symptoms.

H.1 Information Retrieval Cognitive Failures

We identify five failure modes from Technology Research tasks requiring agents to produce comprehensive reports on recent AI developments.

Mode A: Factual Hallucination via Stale Training Data (∼30%). When search tools fail to retrieve current information, models substitute outdated training data *without acknowledging uncertainty*. Table 11 shows a representative case: Claude Opus 4.6 reports Harvey AI's Series D funding figures when the task asks about the recent F-round, with every data point incorrect by a large margin. The same error appears across different scaffolds (both Claude Code and Cursor), confirming this is a model-level issue rather than a scaffold artifact. The model produces no hedging language (e.g., "this information may be outdated"), presenting stale data with full confidence.

Dimension	Ground Truth	Model Output	Error
Funding round	F-round	Series D	Wrong round
Amount	$160M	∼$300M	1.9×
Valuation	$8B	∼$3B	2.7× off
Lawyer users	74,000	Not mentioned	Missing

Table 11: Factual hallucination example: Claude Opus 4.6 on Harvey AI funding data. The model substitutes outdated Series D data for the requested F-round.

Mode B: Imprecise Retrieval (∼35%). The most prevalent failure mode. Models correctly identify the research *area* a question targets and conduct extensive searches, but cannot locate the *specific* papers required by the rubric. For example, on a task about Tool-Integrated Reasoning (TIR), no model found all three required papers (ToolRL, ToRL, ZeroTIR—all from early 2025). The best-performing agent (Cursor + Opus) produced a 16KB TIR survey that demonstrated genuine understanding of the field, but substituted related papers (ReTool, THOR, Tool-Zero) for the specific ones the rubric expected. Models also create their own conceptual frameworks rather than using original terminology—e.g., inventing "precision ceiling" and "generalization ceiling" instead of the papers' actual terms "empirical support set" and "feasible support set."

Mode C: Rigid Search Strategy (∼15%). GPT-5.2 exhibits variable search persistence: while it can make extensive searches (5–75 attempts across tasks), it occasionally declares "tool limitations" and outputs placeholder templates on specific tasks. In the same Docker environment, Claude Opus 4.6 more consistently attempts 30–50 search variations, cycling through arXiv API, Semantic Scholar, OpenAlex, and alternative keywords before writing. Critically, this is scaffold-dependent: GPT-5.2 via Codex scores 0.4 on a task where it scores 0.0 via Claude Code, because Codex's prompting strategy encourages more aggressive tool use. This reveals that search persistence is *elicitable* rather than a fixed model property.

Mode D: Attribution Confusion (∼10%). Models find correct papers but extract incorrect metadata by mixing information across search results. Claude Opus 4.6 correctly identifies the ToRL paper but attributes it to "Tsinghua University & ByteDance" (likely from a concurrent paper) instead of the correct "Shanghai Jiao Tong University," and reports a benchmark-specific accuracy (43.3% on AIME24) instead of the paper's reported average accuracy (62.1%), conflating different evaluation metrics. GLM-5 *inverts* the core finding of a paper, writing that SFT preserves old capabilities and RL enables new tasks when the opposite is true.

Mode E: Positive-Information Bias (∼10%). All models systematically miss negative industry events. A task requiring coverage of AI agent startup failures found that *no model* reported Robin AI's distressed sale despite it being a prominent Series C failure. Models search "AI agent startups 2025" and receive results dominated by successful funding announcements; none proactively searched for "AI startup failures" or "AI startup bankruptcy." Similarly, all models missed niche success stories (e.g., Gamma achieving $100M ARR with only 50 employees) that fall outside the mainstream "agent startup" narrative.

Model Capability Profiles. Table 12 summarizes the distinctive cognitive profiles of each model, revealing that models have complementary rather than uniformly ordered capabilities.

Model	Search	Halluc.	Signature Pattern
Opus 4.6	30–50	High	Fills gaps with stale data confidently
GPT-5.2	5–75	Low	Variable persistence, sometimes outputs templates
Gemini 3 Pro	10–20	Low	Conservative, acknowledges uncertainty
GLM-5	5–10	Med	Conceptual inversion errors
MiniMax M2.5	3–5	Med	Correct direction, wrong details
Kimi K2.5	5–15	Med	Moderate persistence, domain-specific strengths

Table 12: Model cognitive profiles on technology research tasks. Search persistence is measured by typical number of search attempts before writing.

H.2 Cross-Domain Cognitive Failures

Beyond information retrieval, we identify production-specific cognitive failure modes across multiple domains. These failures are invisible to coding benchmarks because they require domain knowledge, subjective judgment, cross-document reasoning, and constraint satisfaction under ambiguity.

H.2.1 Human Resources: Subjective Judgment Failures

Human Resources tasks require agents to evaluate resumes against multi-dimensional hiring criteria, exposing failures in subjective assessment that no coding benchmark tests.

Criterion Explicitness Dependence. Agent performance inversely correlates with criterion subjectivity. Tasks with quantifiable requirements ("5+ years Python experience," "AWS certification required") achieve scores 2–3× higher than tasks requiring holistic judgment ("strong leadership potential," "culture fit"). This suggests models have not learned to *infer* soft-skill indicators from indirect evidence—e.g., inferring leadership from descriptions of project scope, team size, and outcome ownership.

Inconsistent Criterion Weighting. When rubrics specify multiple evaluation dimensions without explicit weights, agents apply inconsistent priority orderings across candidates. The same agent may prioritize "technical depth" for one candidate and "breadth of experience" for another within the same evaluation batch, producing rankings that a human recruiter would flag as internally inconsistent.

Cross-Candidate Calibration Failure. Agents evaluate each resume independently rather than comparatively. When a strong candidate appears early in a batch, subsequent candidates receive inflated scores because the agent has no persistent calibration anchor. This "memoryless evaluation" pattern means that ranking quality degrades with batch size.

H.2.2 Healthcare & Life Sciences: Protocol Reasoning Failures

Healthcare & Life Sciences tasks require precise adherence to study protocols with cascading temporal dependencies, exposing failures in structured reasoning that superficially resemble—but fundamentally differ from—coding logic errors.

State Machine Precondition Semantics. Clinical protocols define visit windows, dose modifications, and adverse event escalation as state machines with preconditions. Agents correctly implement individual state transitions but fail on precondition chains—e.g., correctly computing that a dose reduction is needed but not recognizing that the reduction requires a preceding safety assessment that itself has a 48-hour observation window.

Protocol Terminology Confusion. Clinical research distinguishes between "Protocol Deviation" (minor, reportable) and "Protocol Violation" (major, may require participant withdrawal). Models conflate these terms or apply incorrect severity classifications, with downstream effects on recommended actions. This is a domain-specific semantic distinction that no general-purpose benchmark tests.

Temporal Escalation Logic. Multi-visit study protocols specify escalating responses to repeated events: first occurrence triggers documentation, second triggers dose modification, third triggers study withdrawal. Models correctly handle individual events but fail to maintain *cross-visit state*—treating each visit as independent and never triggering the escalation sequence.

H.2.3 Finance & Investment: Cross-Section Coherence Failures

Investment analysis tasks require producing multi-section reports (market sizing, competitive analysis, risk assessment, financial projections) that must be internally consistent, exposing a failure mode unique to long-form analytical writing.

TAM-SAM-SOM Logical Inconsistency. Agents produce market sizing sections where the Total Addressable Market (TAM), Serviceable Addressable Market (SAM), and Serviceable Obtainable Market (SOM) contain contradictory figures—e.g., a TAM of $50B in the executive summary but $80B in the market analysis, or a SOM that exceeds the SAM. Each paragraph is locally plausible; the inconsistency only surfaces when reading the full document.

Context Collapse: Generic vs. Company-Specific Analysis. Models produce analysis that reads as a *sector* report rather than a *company* report—discussing general industry trends without grounding them in the specific company's financials, competitive position, or strategic choices. The most common manifestation: risk factors that are industry-generic ("regulatory uncertainty," "competitive pressure") without explaining how they specifically affect *this* company's business model.

Hallucinated Market Data. When specific market data is not provided in the input documents, models fabricate plausible-sounding statistics ("the global SaaS market is projected to reach $X billion by 2027") without citing sources. Unlike information retrieval hallucination (Mode A above), this occurs even when the model is not explicitly asked to search—it fills analytical gaps with invented quantitative claims.

H.2.4 Procurement & Operations: Optimization and Constraint Failures

Procurement tasks require multi-attribute optimization under technical constraints, exposing failures in mathematical reasoning and constraint satisfaction that coding benchmarks test only superficially.

Synergy Blindness in Multi-Attribute Optimization. When optimizing across multiple correlated attributes (cost, performance, power consumption, thermal rating), agents optimize each attribute independently rather than jointly. This produces solutions with 26% average cost overruns compared to jointly optimal solutions, because models fail to exploit component synergies (e.g., a slightly more expensive component that reduces cooling requirements and total system cost).

Constraint Specification Misinterpretation. Technical specifications use domain-specific notation that models misread: $\pm$10V tolerance interpreted as $\pm$5V, "Type II" classification confused with "Class II," or "continuous rating" conflated with "peak rating." These are not hallucinations—the model reads the specification but applies incorrect domain knowledge to interpret it.

Infeasibility Recognition Bias. When a procurement problem has no feasible solution (conflicting constraints), the majority of agent responses fabricate a "best effort" solution rather than declaring infeasibility. Models exhibit a strong completion bias—they assume every problem has a solution and will relax constraints silently rather than report that the requirements cannot be simultaneously satisfied. This is particularly dangerous in production, where a fabricated solution may be acted upon.

H.2.5 Finance & Investment: Grounded Critique Failures

Pitch coaching tasks require agents to provide actionable, data-grounded feedback on startup pitches, exposing the difference between structurally correct and substantively useful output.

Template Critique vs. Grounded Analysis. All models produce feedback that follows the correct structure (strengths, weaknesses, recommendations) but fails to reference the specific data provided. A critique stating "your market size claims need better support" is generic; a useful critique states "your Slide 7 claims a $2B TAM but your bottom-up calculation on Slide 12 implies $800M—reconcile these." Models consistently produce the former. This failure is invisible to any benchmark that evaluates structural correctness.

User Category Confusion. When pitch decks present multiple user metrics (total registered users, monthly active users, core power users), models confuse categories—citing "500K users" without specifying which category, or conflating growth rates across categories. This produces feedback that sounds data-informed but references the wrong numbers.

Implicit Question Extraction. Meeting transcripts contain implicit investor concerns (e.g., a question about "team background" that is really probing founder-market fit). No model reliably identifies these implicit questions or addresses the underlying concern rather than the surface question.

H.2.6 Finance & Investment: Scale and Mapping Failures

Beyond information retrieval (Section H), Finance & Investment tasks requiring data synthesis from multiple documents expose additional cognitive failures.

Numerical Data Mapping Errors. When synthesizing data from tables across multiple documents, agents extract correct numbers but assign them to incorrect columns or rows. For example, correctly reading that Company A has revenue of $50M and Company B has revenue of $30M, but swapping them in the output table. This error rate increases non-linearly with the number of data points: tasks with $<$20 data points show $\sim$5% mapping errors, while tasks with $>$100 data points show $\sim$25%.

Authority Hierarchy Misunderstanding. When documents contain conflicting information (e.g., a press release states one figure while an SEC filing states another), models do not consistently prioritize the more authoritative source. In production, domain experts have clear authority hierarchies (regulatory filings $>$ press releases $>$ news articles); models treat all sources as equally credible.

Selective Document Reading. When given a large document set, agents read documents in the order provided and exhibit recency bias—information from later-read documents is weighted more heavily. If critical information appears in an early document that the agent scans quickly, it may be omitted from the final output even though the agent demonstrably "saw" it (evidenced by interaction logs).

H.2.7 Software Engineering: Specification-Implementation Gap

Software Engineering tasks expose a distinctive failure mode where agents produce functional code that satisfies superficial requirements but misses deeper specification intent. For example, agents implement correct navigation structure but omit specified interaction details (e.g., swipe gestures, animation timing). The gap between "code that compiles and runs" and "code that matches the product specification" mirrors the broader production challenge: research benchmarks test functional correctness, while production demands specification fidelity.

I Representative Task Examples

We present one representative task from each of the six O*NET domains to illustrate the production complexity *AlphaEval* captures.

Human Resources: Resume Screening (hunter-ai-1). The agent is given a job description for an AI Scientist Mapping Intern role together with department-specific hiring criteria (e.g., minimum internship duration, university tier, prior AI experience) and must screen 24 candidate resumes provided as PDF and JPEG files. The agent must select exactly six candidates to advance to a first-round interview and output the chosen candidate IDs as a Python list in a designated answer file. Evaluation compares the agent's selections against the ground-truth shortlist using precision, recall, and F1 score, with a passing threshold of F1 ≥ 0.6.

```
name: "AI Resume Screening - Mapping Dept"
category: hr
difficulty: hard
evaluation:
  type: code_exec
  rubric_script: .eval/rubric.py
agent:
  timeout_seconds: 600
  max_turns: 100
```

Finance & Investment: Segment Research Report (segment-research-1). The agent acts as a senior industry research analyst and must produce an investment-grade Segment Research report from a startup's business plan PDF. The report must follow a prescribed eight-section template covering industry definition, TAM–SOM–SAM market sizing (with both top-down and bottom-up methods), product landscape, customer typology, key success factors with quantitative thresholds, three to four competitive case studies with GTM strategy analysis, and a technology deep-dive including patent barriers and commercialization feasibility. A reference template is provided. Evaluation uses LLM-as-a-Judge to assess structural completeness, analytical depth, data accuracy, and adherence to the template format.

```
name: "Segment Research Report"
category: research
difficulty: hard
evaluation:
  type: code_exec
  rubric_script: .eval/rubric.py
agent:
  timeout_seconds: 1200
  max_turns: 150
```

Procurement & Operations: BOM Cost Optimization (yuhe-1). The agent receives a catalog of 2,000 board cards with detailed specifications and pricing in an Excel file, a field-structure CSV defining column semantics, and a natural-language procurement requirements document specifying functional constraints. It must select an optimal combination of board cards that satisfies all functional requirements while minimizing total procurement cost, breaking ties by fewest cards and then by ascending card ID. The output is a structured cost breakdown listing each selected card's ID, quantity, unit price, and subtotal. Evaluation uses a rubric script that verifies constraint satisfaction and compares total cost against a known optimal solution.

```
name: "BOM Board Card Procurement Optimization"
category: optimization
difficulty: hard
evaluation:
  type: code_exec
  rubric_script: .eval/rubric.py
agent:
  timeout_seconds: 600
  max_turns: 100
```

Software Engineering: Poetry Mini-Program (miniprogram_poetry). The agent must implement a complete WeChat Mini-Program for classical Chinese poetry appreciation, following a detailed 200-line product requirements document that specifies page layout (gallery-style hand-drawn illustration aesthetic), navigation architecture (four-tab bottom bar), core features (AI text-to-speech recitation with speed and voice controls, waterfall-style poem browsing, a community forum for user posts, and a personal collection manager), and interaction patterns (modal dialogs for content creation, clipboard-based sharing). The deliverable is a fully functional codebase. Evaluation combines automated UI testing to verify page navigation and component rendering with rubric-based checks on code structure and feature completeness.

```
name: "Poetry Appreciation Mini-Program"
category: miniprogram
difficulty: hard
evaluation:
  type: miniprogram
  rubrics: [10 weighted UI test points]
agent:
  timeout_seconds: 1800
  max_turns: 100
```

Healthcare & Life Sciences: eCRF Visit Window Calculation (linchuang-1). The agent operates within a simulated eCRF (electronic Case Report Form) system for a Phase III non-small-cell lung cancer clinical trial. Given a JSON-based visit-window rule configuration and a patient's screening visit date, the agent must compute target dates and permissible windows for subsequent treatment visits, determine which visits can be calculated given available data, and perform cascade-impact analysis when an actual visit date deviates from the target. The output must show explicit calculation formulas and all dates in YYYY-MM-DD format. Evaluation uses a rubric script that checks date arithmetic correctness, proper application of window offsets, and accurate cascade reasoning.

```
name: "eCRF Visit Window Calculation"
scenario: linchuang
evaluation:
  type: code_exec
  rubric_script: .eval/rubric.py
files:
  - files/JSON_eCRF_CORE.md
  - files/CRF_visit_window_spec.pdf
```

Technology Research: AI Agent Startup Landscape (jiqizhixin-1). The agent acts as a professional AI industry analyst and must produce a comprehensive research report on the 2025 landscape of AI agent startups. The report must cover four dimensions: agent startups that raised over $100M (with vertical categories such as legal, coding, and search), startups that failed or were acquired along with root-cause analysis, large-tech acquisition activity targeting agent companies, and shifts in Anthropic's Claude adoption within the YC ecosystem. All claims must be supported by specific figures such as funding amounts and market sizes, with at least reliable source references. Evaluation uses LLM-as-a-Judge to assess factual accuracy, analytical depth, data substantiation, and structural completeness.

```
name: "AI Agent Startup 2025 Landscape"
category: research
difficulty: hard
evaluation:
  type: code_exec
  rubric_script: .eval/rubric.py
agent:
  timeout_seconds: 1200
  max_turns: 150
```

J Practitioner Survey: AI Product Deployment Challenges

To ground *AlphaEval* in real practitioner needs, we conducted a mixed-methods survey of 27 AI product companies affiliated with a startup accelerator (response rate: 54% of 50 invited). The survey combined structured questionnaires (15 questions) with open-ended text analysis, conducted from December 2025 to January 2026.

Respondent Profile. Companies span four development stages: concept verification (14.8%), internal pilot (14.8%), early commercialization (48.1%), and scaled deployment (22.2%). The majority target enterprise clients (59.3%), with consumer-facing products at 29.6%, with the remainder targeting internal employees or niche verticals. All companies support text input (85.2%), with growing adoption of image (55.6%), structured data (48.1%), audio/video (44.4%), compound documents (40.7%), and code (37.0%).

Evaluation Infrastructure Gap. Current evaluation practices reveal significant infrastructure deficits:

- 25.9% of companies have *no explicit evaluation criteria*
- 33.3% rely solely on small sets of "golden samples" with manual inspection
- Only 11.1% have established structured test datasets with automated evaluation pipelines
- 22.2% have closed-loop A/B testing feedback (primarily among scaled companies)

Core Technical Challenges. The most frequently cited challenges (multiple selection, N=27):

- Output instability / inconsistency: 59.3%
- Instruction-following failures in complex scenarios: 51.9%
- Hallucination / factual errors: 40.7%
- Low inference efficiency / high token costs: 40.7%
- Long-context processing / memory loss: 33.3%
- Multi-turn dialogue degradation: 22.2%
- Safety / compliance risks: 14.8%

Confidence in Model Updates. When asked "After each model update or prompt modification, are you confident the new version is better?":

- 25.9% reported high confidence (with established evaluation systems)
- **63.0%** reported low confidence (lacking reliable evaluation mechanisms)
- 7.4% reported no confidence at all
- 3.7% reported the question was not applicable (not yet at frequent update stage)

Testing Resource Allocation. 70.4% of companies rely on *developers performing testing as a side task*, consuming development time. Only 18.5% have dedicated testing personnel with substantial human review investment. 11.1% depend entirely on user feedback post-deployment.

Top Evaluation Needs. Open-ended text analysis (keyword extraction and co-occurrence network) identified three demand themes:

1. **Automated evaluation platform construction** (weight: 0.47): automated problem localization, iteration recommendations, and priority ranking
2. **Objective evaluation standards** (weight: 0.24): reliable product quality verification and performance benchmarking
3. **Cost and efficiency optimization** (weight: 0.18): reducing testing costs, improving inference efficiency, and token cost control

Survey Instrument. The questionnaire comprised 15 items covering product information (Q1–Q5), technical status (Q6–Q11), and evaluation needs (Q12–Q16). Key questions included: development stage (single choice), target customer type (single choice), input modalities supported (multiple choice), evaluation infrastructure maturity (single choice, 5-level scale from "no criteria" to "closed-loop A/B testing"), core technical challenges (multiple choice, 7 categories), confidence in model updates (single choice, 4-level scale), testing resource allocation (single choice, 3-level scale), preferred evaluation methodologies (multiple choice), required execution environments (multiple choice), and willingness to share anonymized test data.

These findings directly motivated the design of *AlphaEval*: the 63% low-confidence rate in model updates underscores the need for reliable automated evaluation, the diversity of input modalities (42% PDF, 21% structured data) informed our task composition, and the demand for automated evaluation platforms (weight 0.47) validates our requirement-to-benchmark construction framework.